Jashore University of Science and Technology

Jashore-7408, Bangladesh

Campus Based Donation Management System

Submitted By

Ramjan Ali Kha

Student ID: 200114

Registration Year: 2020-21

Supervisor

Dr. Mohammad Nowsin Amin Sheikh

Assistant Professor

Department of Computer Science and Engineering

This project report was submitted in partial fulfillment of the

Requirements for the BSc.Engg Degree

In **Computer Science and Engineering.**

April 2026

Jashore University of Science and Technology

Jashore-7408, Bangladesh

This project titled **Campus Based Donation Management System** submitted by **Ramjan Ali Kha** for final defence for the degree of B.Sc. in Computer Science and Engineering on April 27, 2025.

Signature of the Student

Ramjan Ali Kha

Signature of the Supervisor

Dr. Mohammad Nowsin Amin Sheikh

Acknowledgments

I want to thank **Almighty Allah**, the Creator and the Sustained of the word, for providing us with the opportunity, capacity, energy, spirit, and patience to finish the report.

I want to express my sincere gratitude to Dr. Mohammad Nowsin Amin Sheikh, assistant professor at Jashore University of Science and Technology and my respected educator and supervisor. His unwavering advice and direction were crucial to the success of this project. In order to make this project writing as perfect as possible, he was constantly offering valuable assistance. The completion of the report would not have been possible without his extensive knowledge, insightful ideas, boundless enthusiasm, invaluable advice, assistance, and suggestions. I am honored to have had the chance to work and study under his supervision.

I also want to express my gratitude to the honorable faculty of the Department of Computer Science and Engineering. I appreciate their support and helpful criticism, which enabled me to write my project report more skillfully.

My friends and family have been a major source of inspiration and passion for me, and for that I am incredibly grateful. All of their love, kindness, and support have been invaluable to me during this time.

I would want to thank my parents and family for their unwavering support and motivation. I am grateful for your understanding, love, support, and patience, all of which inspired me to complete my studies.

Table of Contents

List of Figures

List of Tables

List of Acronyms

List of Acronyms.

Acronyms	Full Form
DMS	Donation Management System
UI	User Interface
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
ORM	Object-Relational Mapping
REST	Representational State Transfer
HTTPS	Hypertext Transfer Protocol Secure
SQL	Structured Query Language
EF	Entity Framework
RBAC	Role Based Access Control
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
TypeScript	Programming Language
React	JavaScript Library
Redux	State Management
.NET	Framework
C#	Programming Language

Abstract

Any charitable effort will require efficient coordination of the money collected, effective distribution of resources, rigorous supervision of the volunteers and proper handling of finances. There are many organizations who still rely on old-fashioned record keeping systems that include paper files and manual operations. Weaknesses associated with such methods can be easily identified while participating in different fundraising efforts in the campus ranging from 2024 flood relief fundraiser in Sylhet to the fund-raising event aimed at helping our sister, Atika (CSE 17-18), who recently passed away due to her incurable disease. Our experience during these fundraising activities is the reason behind developing a Donation Management System, which will be discussed in detail in this project. DMS (Donation Management System) includes everything that is related to the donations.

The system uses a multi-tiered software architecture. ASP.NET Core Web API is used to create the backend whereas React/TypeScript creates the frontend and the Entity Framework Core database-first approach is used to design the database. Passwords are protected using several security measures that include Bcrypt password encryption, antiforgery tokens and server side input validation. Users can make easy, secure donations through a payment gateway that works for all the devices.

The platform's rich feature set greatly enhances the ability to manage campaigns by providing the ability to process online and physical donations using OTP authentication, role-based access control, collaboration of distributed volunteers, and machine learning techniques to perform sentiment analysis on testimonials. By successfully replacing fragmented human steps with accurate automation, you will create lasting donor confidence, assuring full visibility and accountability of operations.

Chapter 1

Introduction

Chapter 1: Introduction

1.1 Overview

The **Campus-Based Donation Management System** is thus described as a web-based software solution that is aimed at the automation of the process of administration of tasks pertaining to philanthropic activities of an organization [1, 2]. The introduction of multiple roles of administrator, donor, and volunteers to the system eliminates the majority of the issues associated with paper-based solutions.

Functionally, the application works under three main purposes:

- **Financial Management:** Real-time monitoring of donations made, transactional payments through SSLCommerz [3] payment gateway on the website, and verification of physical donations through OTP verification for registration purposes.
- **Process Management:** Process life cycle management for campaigns, meritocracy of volunteers, and the transaction voucher authorization chain in place of ad hoc process.
- **Predictive Analysis:** Machine learning system within the application predicting donation probability, anomaly detection, and donor attrition prediction.

In terms of architecture, the platform conforms to a modern tri-layer structure. **React with TypeScript** [4, 5] is used on the presentation layer to provide responsiveness and type-safety on the user interface side. The business layer is implemented with an **ASP.NET Core Web API** [6, 7], and **Microsoft SQL Server** is used for data storage through Entity Framework Core [8, 9]. JSON Web Token (JWT) authentication, BCrypt hashing algorithm, and Role-Based Access Control (RBAC) [10] are employed for the implementation of system-wide security enforcement.

1.2 Background

Traditional methods of managing donations, coordinating campaign efforts and handling volunteers for charity organizations and campus-based donation drives are manually intensive, paper-based systems or at best loosely coordinated digital platforms. Clearly, solutions that may work at a smaller level are not appropriate for operations of growing complexity. The following weaknesses have been identified both through literature reviews and empirical observations and were directly used in setting out the system requirements for DMS:

- **Tracking latency:** There is a considerable lag in monitoring the incoming donations or progress of the campaign. Information on the status of donation drive and campaign activities is always delayed by some hours or some days.
- **Data Redundancies:** The same record entered in separate sheets is duplicated and leads to redundancies in data entries that make subsequent corrections laborious and subject to further discrepancies.
- **Operational Weaknesses:** There are no systematic means of tracking volunteer contributions. It is impossible to quantify the hours volunteered, the tasks done or the resources used.
- **No Analytics:** No analytics make it hard to know how successful a campaign is, what trends there are in donor behavior, and why involvement is declining.
- **Barriers to Communication** The communication processes with the organizations and their donors are often not structured enough so it is impossible to develop proper feedback mechanisms.
- **Transparency Concerns:** Lack of transparency in the application of funds by an organization will lead to a constant decrease of trust and hence the future involvement by the donors may become less probable.

- **Funding Responsibility:** There's no audit trail and there's no way to know that the money being spent is aligned with organizational goals.
- **Donor Invisibility:** Donors to the organization are never told about any performance indicators related to progress on campaigns or disbursements thus creating a disincentive for donations.
- **Physical Asset Tracking Failure:** Contributions of in-kind goods, like clothes, food, etc., are hardly documented. Therefore, there is a deficiency in accounting for assets.
- **Weaknesses in Cost Authorization Control:** Campaign expenses lack systematic cost authorization control, posing problems when auditing costs associated with operations.
- **Uncontrolled Financial Transactions:** The lack of financial transaction control protocols and reserved fund controls poses increased risks of financial misappropriation, which also reduces auditability.
- **Absence of Forecasting and Analysis Functionality:** Current systems are reactive, storing transaction data only; however, they do not offer forecasting capabilities, transaction anomaly detection, and donor abandonment prediction.
- **No Merit System for Volunteers:** Absence of a well-developed merit system and reward for accomplishments demotivates volunteers and hinders their retention.
- **Manual Stakeholder Communication:** Important stakeholder communications are conducted manually, or even worse, they are omitted entirely, reducing efficiency and effectiveness of communication.

The increasing speed of technological change and growing demands for advanced user interfaces make the introduction of centralized automation solutions essential. The proposed Donation Management System is a solution that will help solve these issues by offering an advanced, automated solution, ensuring high data security and financial transparency alongside universal accessibility.

1.3 Motivation

The motivation behind such an idea is the constant occurrence of problems within the administration of the process of philanthropic management itself. In other words, there are certain problems arising consistently in the context of charities performed by organizations, in particular, charities performed in educational institutions, including:

- Reconciliation of donations made in cash, online, and via mobile banking.
- Control of volunteering and involvement of participants in the process of donation.
- Campaign success assessment and feedback collection for improving it.
- Information dissemination about the progress of ongoing campaigns and how money is allocated to it.
- Financial transaction and donor safety.

In order to solve all the problems listed above, it is necessary to create a framework for an extendable application. Such a decision will help solve all the aforementioned problems, but will also give a basis for further development and improvement of our solution [11]. At the same time, modern technology allows implementing applications that are stable, secure, and highly satisfactory to users [12].

1.4 Problem Statement

There are a number of challenges for companies managing their philanthropy operations related to manual operations and data management paradigms [13, 2]. These problems are:

- **Problem with Real-time Monitoring of Campaigns:** The lack of timely information means that administrators and donors are unable to monitor the progress of the campaign and the rate of contributions in real time.

- **Security Issues for Manual Donations:** Manual processes used for receiving cash donations or donations in any other form are inefficient, faulty and insecure.
- **Problems of Task Distribution and Monitoring:** Central systems are not enough, so task distribution and volunteer monitoring are problematic.
- **Problems Related to Data Consistency:** Inconsistency in data is caused by manual recording systems to develop accurate reports.
- **Absence of Analytical Tools:** There is a lack of tools within administrative organisations to extract information on donors' action, behaviour and testimonial sentiments.
- **Security and Privacy Threats:** Unprotected databases are under a serious threat of breaching the confidentiality of financial information.
- **Hidden Operations:** Donors' high correlation with accountability of funds renders existing systems unable to utilise transparent tools.
- **Lack of Integrated Financial Management:** The existing approach fails to integrate bookkeeping functions to align incoming donations with financial statements.

The above weaknesses demonstrate the necessity of developing an automated web-based framework to ensure transparency within philanthropy management.

1.5 Questions Addressed by the Project

These questions will drive the design process of the proposed system by highlighting the weaknesses of the current approach to managing donations:

- How well can a unified web solution outperform the traditional methods used for managing donations?
- What benefits does a role-based approach have when applied to multiple users involved in a donation campaign?

- How does real-time monitoring affect campaign visibility, customer satisfaction, and overall trust?
- How does including a secure payment solution make the process of making a donation easier?
- How can analyzing donor feedback help us understand how a particular campaign affected its audience?
- What methods should be applied for protecting confidential data from unauthorized access?

1.6 Objectives

Guided by the aforementioned research questions, the project is set to achieve the following primary goals:

- **To create a cohesive web portal** that combines features for donations, campaigns, volunteering, and reporting within a single platform.
- **To apply role-based permission control** that assigns varying levels of privileges to Administrators, Donors, Volunteers, and Campaign Creators.
- **To enable real-time tracking of donations and campaigns** that ensures transparency and instant updates for all users.
- **To incorporate secure payment gateways** using the SSLCommerz system for smooth transactions.
- **To provide an optimal user interface** with responsive design compatible with various devices.
- **To ensure information security and privacy** by employing strong authentication, encryption, and server validation procedures.
- **To conduct sentiment analysis** of testimonials left by donors as a metric of campaign success and donor satisfaction.
- **To generate comprehensive reports** that facilitate evidence-based decision-making and performance measurement.

1.7 Contributions

The development and implementation of the Donation Management System (DMS) offer a number of unique contributions to the field of philanthropic management, especially within the context of the academic and campus setting. With the help of a fully-integrated, end-to-end managed system, the current project helps address an urgent need for the transition from spreadsheet-based tracking to timely transparency. Key contributions of the work include:

- **Unified Architecture:** The creation of an all-encompassing multi-tiered web application for managing various types of donations, volunteer participation, and campaigns, thus reducing any unnecessary administrative burdens of interdepartmental communication to the minimum.
- **Improved Security and Transparency:** Implementing an efficient OTP process for verifying in-kind contributions, as well as a robust framework for financial monitoring that greatly increases trust and accountability of the process.
- **State-of-the-Art Analysis:** Utilizing advanced machine learning for assessing sentiments expressed by donors in testimonies and using these findings to optimize campaign effectiveness through iterative analysis and improvement of fundraising strategies.
- **Secure Environment Framework:** Ensuring a safe operating environment via effective Role-Based Access Control (RBAC), data encryption, and compatibility with proven digital payment solutions. These steps ensure that threats like data theft and transaction fraud are rendered ineffective.

1.8 Organization

The project report consists of six sections, each of which corresponds to a particular phase of the project development life cycle. These sections include:

Chapter 1 Introduction. Introduces the project description, background information, motivations, problem statement, research questions, goals, and outline of the report.

Chapter 2 Background and Technologies Used. Outlines current donation management systems, analyzes underlying technologies used, and identifies functionality gaps that should be addressed by the proposed system.

Chapter 3 Methodology. Provides information on the methods used for designing the system architecture, its development, testing, deployment, and security implementation.

Chapter 4 System Architecture and Design. Describes the frontend and backend designs, database design, and corresponding design artifacts (ER diagram, Use Case Diagram, Schema, and C4 Model).

Chapter 5 Implementation. Explains the system architecture design, implementation of key components, and technologies used, along with relevant screenshots of the application user interfaces.

Chapter 6 Conclusion and Future Work. Summarizes the results obtained and outlines further possible improvements and developments.

Chapter 2

Background and Related Technologies

Chapter 2: Background and Related Technologies

2.1 Overview

The increasing digitization of business processes has brought about numerous implications for the charitable sector. There has been a notable shift in the approach towards the management of donations from traditional paper-based systems to computerized methods aimed at integrating campaign management, volunteer management, and accounting services. This chapter provides an in-depth analysis of the current models used for donation management, discusses the main digital tools currently being utilized, and highlights the technological basis of the proposed Donation Management System (DMS). The chapter ends with the identification of some shortcomings addressed by the DMS.

2.2 Conventional Donation Management Techniques

Traditional systems for managing donations continue to rely primarily on manual procedures such as physical documentation, face-to-face collection, spreadsheet-based accounting, and ad hoc communication [1, 14]. While these processes are easy to implement and relatively economical at first glance, they carry a number of well-known weaknesses when scaling up:

- **Inadequate Real-Time Monitoring:** The lack of an automated system for data entry means that there will always be some delay between donations being made and the corresponding data being recorded in the organization's database. Reporting on campaign performance could be delayed by several hours or even days.
- **Potential for Data Quality Issues:** Human error can lead to misspellings, loss of receipts, duplicate entries, among others, thus reducing the accuracy of further analysis.
- **Challenges with Volunteer Coordination:** Without centralized management systems, it would be difficult to organize and supervise volunteers.

- **Insufficient Analytical Capabilities:** Spreadsheets provide limited capabilities for detecting trends or assessing the effectiveness of campaigns.
- **Exposure to Security Threats:** Files containing sensitive information are likely to be left unencrypted on network drives and personal computers.

2.3 Existing Digital Donation Management Solutions

At present, there exist various classes of platforms aimed at fulfilling particular subgroups of requirements described above. Based on an analysis of selected solutions, it can be stated that:

As-Sunnah Foundation: The religious humanitarian foundation operates using its own digital platform geared towards the South Asian market of donors. It utilizes MFS services such as bKash and Nagad for swift mobilization of funds, along with using a system of visual evidence-based transparency measures for maintaining the trust of donors [15]. However, while the platform is effective in the context of its operations, it remains proprietary and thus cannot be used by other organizations due to the lack of extensibility.

Donorbox: This is a web-based solution providing integration of payment processing, donor profile management, as well as customizability of donation forms [13]. The solution allows for monitoring of transactions and setting up automated processes for donor communication. Thus, it is appropriate for organizations that rely primarily on money-based donations received through their website.

GiveWP: As a WordPress plugin designed to be modular in nature, GiveWP enables customizable forms and is fully compatible with WordPress-based content management systems. For nonprofits working outside the WordPress environment, this technology holds minimal value.

Network for Good: A one-stop fundraising coordination system that deals with the management of donors and volunteers along with analytics. Although Network for Good functions as a comprehensive package, the fees associated with it make it unaffordable for smaller nonprofit organizations.

Blockchain-Powered Technologies: Newer technologies based on blockchain have shown promising capabilities of recording philanthropic transactions in a transparent yet secure manner through audit trails [16]. However, most of the blockchain-powered donation systems are still in their nascent phases.

2.3.1 Critical Limitations in Existing Solutions

A comparative assessment of the discussed platforms identifies certain limitations that hinder their use for philanthropic operations on college campuses:

- **Customization Shortcomings for Institutional Needs:** Commercial fundraising platforms are usually developed for nonprofit use in general, with no customization to fit the particular processes associated with student-to-student transactions that occur within college settings.
- **Limitations Related to Budgets:** The subscription and license fees associated with these platforms make their use prohibitive for college organizations operating on tight budgets [17].
- **Lack of Interoperability with Existing Tools:** The discussed commercial platforms typically fail to interface with existing tools, such as student data registries and financial tracking systems employed by universities.
- **Lack of Predictive Analytics:** The analyzed platforms rarely provide any features related to the prediction of future trends, such as potential losses or donations.
- **Failure to Consider In-Kind Donations:** Most platforms ignore physical and in-kind gifts, which are a part of campus-based fundraising efforts.
- **Insufficient Control Over Funds and Expenses:** There are no features allowing one to manage expenses and withdraw funds according to specific rules.

2.4 Role-Based Access Control in Management Systems

Role-Based Access Control (RBAC) is one of the fundamental authorization mechanisms for multi-user digital systems, allowing fine-grained access control among different types of users [10]. In relation to the DMS, the following are the purposes that RBAC fulfills:

- **Data Protection Implementation:** Access to confidential financial information and personal details is controlled according to the type of user, thus restricting exposure to only authorized roles [18].
- **Enhancing User Interface Design:** Role-specific interface configurations provide users with features relevant to their roles, thus decreasing mental overhead.
- **Integrity Assurance:** Permission enforcement on all incoming requests via middleware prevents unauthorized modifications, even when requests attempt to bypass client-side controls.
- **Audit Log Creation:** Each activity performed in the system is linked to a particular user and role, thus creating a verifiable audit trail.

A hierarchical four-level RBAC structure is used in the Donation Management System to define the following roles:

- **System Administrators:** This group of users has full access to the system, from user management to system settings, finances, and content moderation.
- **Donors:** Can carry out donations, view ongoing campaigns, manage their own profiles, write testimonials, and obtain receipts.
- **Volunteers:** Limited to reporting activities, recording hours, tracking their own progress, and visualizing their achievements.

- **Campaign Creators:** Can manage campaigns from creation to closure, post updates about the campaign's progress, monitor fundraising trends, and communicate with stakeholders.

2.5 Robust Payment Gateway Architectures

A robust online payment processing system is a must-have component in a contemporary philanthropy platform, requiring solid cryptographic protection, transaction verification, and fraud prevention. For the DMS implementation, SSLCommerz, a prominent South Asian payment aggregator [3], was chosen on account of its comprehensive capabilities list:

- **Multiple Payment Instruments Support:** The gateway supports a variety of payment methods, such as credit and debit card payments, mobile money (bKash, Nagad), and online banking systems, integrated via a single point of integration.
- **Compliance with Cryptography Standards:** All transactions go through end-to-end encryption and PCI-DSS standard compliance, ensuring data privacy and security [19].
- **Asynchronous Transaction Verification:** After each successful or failed transaction, the gateway sends callback messages to the server that allows automatic synchronization of transaction status.
- **Audit Trail of Transactions:** Every transaction is recorded in sufficient detail to perform subsequent financial reconciliation and audit operations.

2.6 Advanced Sentiment Analysis and Testimonial Moderation

Sentiment analysis, a domain under natural language processing (NLP), makes it possible to automate the process of extracting evaluative signals from raw text data [20]. The use of sentiment

analysis on donor testimonials allows for deriving quantitative measurements of stakeholder satisfaction and campaign effectiveness without requiring any content inspection. In the context of the DMS, the following tasks are performed by sentiment analysis:

- **Assessment of Campaign Effectiveness:** Summarized sentiment values of testimonials can be used as a quantifiable measurement of donor sentiments about certain campaigns and initiatives of the organization.
- **Automatic Content Moderation:** Testimonials marked with negative sentiment will be automatically flagged for administrative approval prior to their publication. Thus, subjectivity of moderators will be reduced.
- **Monitoring of Long-term Satisfactions:** Changes in distribution of sentiment values over time may be used for tracking changes in donor satisfaction.
- **Integration into Decision-making:** With sentiment analysis combined with financial information about the organization, the decision-makers will have a comprehensive evaluative tool for refining campaign strategies.

Testimonials with negative sentiment are automatically flagged by the DMS before they go live, ensuring quality of publicly available information while reducing the burden of manual moderation work [12].

2.7 Modern Technologies Utilized

Choices of technologies used within the DMS were influenced by criteria including maturity level, scope of the ecosystem, and relevance to the particular problem domain [11]:

- **Frontend:** Presentation layer utilizes React with TypeScript [4, 5, 21], along with Redux Toolkit for state management and Tailwind CSS for styling.

- **Backend:** ASP.NET Core Web API is used on the server side in accordance with REST service architecture principles; the application has distinct controller and service layers.
- **Database:** Relational storage relies upon Microsoft SQL Server, while Entity Framework Core acts as the ORM layer allowing abstraction of database interactions in typed C# language [8, 9].
- **Authentication:** Stateless session management involves JWT bearer tokens with OTP verification sent by email or SMS as second-factor verification means [22, 23].
- **Security:** User passwords use BCrypt hashing [24]; form submissions include CSRF protection [25]; and inputs provided by users are validated and sanitized before being submitted to the database [19].

2.8 Advanced Functionality and Features

In addition to basic functions for collecting and managing donations, the DMS contains a number of advanced modules which seek to fill in some deficiencies present within other similar software solutions. The following is an outline of such capabilities, which form the key differentiating factors of our suggested DMS.

2.8.1 Tracking of Physical Donations

The absence of proper tracking and logging of in-kind and physical donations remains one of the main limitations of commercial donation management systems. To counteract this limitation, the DMS has a built-in tracking functionality [14]:

- **One-Time Password-Based Identification of the Donor:** Before any physical donations can be recorded, the donor is identified using a one-time password sent via SMS message. This reduces the possibility of errors and fraud.

- **Cataloging of Assets:** Every donated asset undergoes systematic documentation, including all of its descriptions, valuations, and time stamps.
- **Generation of Logs:** All events of physical donations result in creation of records suitable for financial and compliance audits.
- **Monetary Ledger Synchronization:** Physical donations are registered in the monetary donation ledger to ensure overall completeness of statistics.

2.8.2 Expense Vouchers and Financial Governance Mechanisms

Formal processes for expenditure recording and expense voucher authorizations represent the fundamental financial governance tools implemented by the DMS:

- **Expenditure Authorization and Documentation:** Campaign expenditure transactions must be authorized formally before fund release and include supporting documentation.
- **Vouchers Attachment:** Receipts and related transaction evidence can be attached to voucher entries.
- **Vouchers Status Monitoring:** Each expenditure goes through the approval workflow stages of being submitted, pending approval, approved, and rejected.
- **Ledger Update:** Upon approval, an expenditure automatically becomes reflected in relevant campaigns' financial statements.

2.8.3 Fiscal Disbursements and Reserve Fund Governance

Control measures governing the disbursement process and reserve funds management ensure the financial discipline of an institution:

- **Disbursements Workflow:** All withdrawals go through a formal request and subsequent approval stages.

- **Reserve Fund Dashboard:** Information on current reserve fund balances and transactions is publicly accessible to all participants.
- **Security Features:** Built-in controls prevent any misuse and unauthorized depleting of funds.
- **Automated Notification Infrastructure:** Interested parties are notified via automated email notifications once withdrawals are approved or there are changes in reserve fund status.

2.8.4 Predictive Analysis Using Machine Learning

The inclusion of machine learning functionality allows for evidence-based strategic planning and proactive risk management [26, 27]:

- **Demand for Donations Forecasting**: SSA-based time series modeling predicts future demand levels of donations over customizable prediction windows.
- **Donor Attrition Prediction**: Binary classification techniques predict donor disengagement risk based on recency, frequency, and monetary characteristics of donors.
- **Anomalies Detection**: Statistical outlier detection techniques detect anomalies in donation transactions and patterns.
- **Campaign Strategy Optimization**: Predictive analysis is used to optimize campaign strategy planning and inform resource planning.

2.8.5 Automated Communication Infrastructure

Effective communication between donors and the organization plays an essential role in achieving loyalty from donors and trust in the organizations. The charity organizations struggled with ineffective communication and manual alerts. Effective communication was hindered by late responses and the lack of transparency. This challenge is solved by using the fully automated

and asynchronous form of communication provided by the Donation Management technique. This module ensures effective communication among all involved parties while eliminating the burden on the institution's employees to perform the same task manually. The effective communication module consists of:

- **Email Notifications:** Automated distribution of emails confirming donations, campaign status, volunteer appointments, and administrative decisions. These emails are formatted using HTML templates to ensure dynamic generation and promote institution brand awareness. Additionally, including impact reports in the email strengthens the relationship between donors and the institution.

- **SMS Communications:** Provisioning of OTP via SMS for authentication purposes and quick alert dissemination. Integration with SMS gateway guarantees that time-sensitive messages are received instantaneously, which is important when tracking in-kind donations. It is worth mentioning that such type of communication channel is also highly effective during volunteer deployment campaigns in emergency situations.

- **Event-Based Notifications:** Event-based message queuing mechanism was implemented into the notification subsystem architecture. Message sending is performed by publishing requests onto the queue of background workers instead of blocking application execution thread to ensure temporal coordination between sending communications and triggering activities.

- **Notification Preferences Management:** The system gives users ability to customize their communication preferences. It allows to specify desired type of notifications and subscribe/ unsubscribe from receiving certain kinds of notifications to ensure better privacy.

- **Logging and Tracking:** Every sent message is properly logged in the relational database along with delivery status and any other pertinent information about interaction.

Chapter 3

Methodology

Chapter 3: Methodology

3.1 Overview

This chapter provides an overview of the implementation process involved in the design, development, testing, and security of the Donation Management System. Instead of using a strict waterfall approach, the task went through numerous iterations starting from collecting the initial set of requirements, implementing functionality, testing, and returning to fixing the found issues. Such an iterative workflow has influenced most of the choices outlined in subsequent sections.

3.2 System Design

Fundamentally, the system consists of three major layers: the frontend client which the user sees, the backend API responsible for handling logic and data processing, and a relational database underneath for persisting data. The ability to keep those layers separate became essential when working on new features since changes in one layer seldom required changes in other layers [11].

3.2.1 Important Considerations

Several critical points that had to be met before writing even a single line of code:

- **Modular design:** The ability to integrate new functionality, for instance, a tracking module based on blockchain technology, without interfering with existing functionality.
- **Tolerant towards faults:** The payment gateway might take too long to respond, the database sometimes deadlocks, etc.
- **Scalability:** The load on the campaign can increase drastically during disaster relief operations. Both the web server tier and the database tier were designed independently such that they can scale horizontally without affecting each other.

3.2.2 Frontend Design

To implement the frontend, we decided to use the Single Page Application (SPA) framework built with React and TypeScript [5]. This is because TypeScript helps identify programming errors early on and saves us from wasting precious time debugging them in the future. Some of the core design decisions made include:

- **Responsive Layouts:** To ensure the user interface works on all devices, our design leverages CSS Flexbox, Grid, and Tailwind utility classes with media queries. This enables seamless transitions and ensures that users have a consistent experience on different form factors irrespective of the screen orientation.

- **Reusable Components:** We opted to develop reusable React components to reduce code duplication and make it easier to add new functionalities. For instance, the dashboard features several reusable components such as the DataTable, Modal, and ChartWrapper.

- **Centralized State with Redux:** Redux Toolkit manages global state management. The store is split into slices per features, such as *authSlice*, *donorSlice*, and *adminSlice* [21]. Thunk middleware is responsible for handling any async actions, like API calls. It means that the components become simpler due to predictable state management, ensuring an easy debugging process.

- **Accessibility (a11y):** WCAG requirements were considered while developing the website. Therefore, we added required ARIA attributes, a high-contrast mode, and ensured that any interactive element was accessible only using keyboard navigation. Thus, we follow inclusive design principles to ensure equal access to the platform for every future donor and volunteer.

Table 3.1 gives a quick overview of how the frontend codebase is organized into layers.

Table 3.1: Frontend Component Categorization

Component Layer	**What It Does**
Presentation Components	Stateless pieces responsible only for rendering buttons, text, cards, and similar visual elements.
Container Components	Stateful wrappers that talk to APIs, subscribe to global state, and coordinate side effects.
Routing Module	Built on React Router DOM; handles page navigation and protects routes that require authentication.
Utility Services	Shared helpers for formatting (dates, currency) and an Axios-based HTTP interceptor layer.

3.2.3 Backend Architecture

Regarding the backend, ASP.NET Core Web API has been selected for developing a REST-style compliant service. The architecture has been developed with the help of several layers, as mentioned below:

- **Controllers:** They are responsible for processing requests on the edges of the system. Every request has to be validated and sent to the appropriate service for generating the response message. A controller has been designed for every functional unit of the application, such as Authentication, Campaigns, Donations, Payments, Testimonials, Volunteers, Vouchers, and Withdrawals.

- **Services:** They include all functionalities of the business layer, which may consist of implementing machine learning models, JWT authentication, payment gateway, emails and SMS sending services, volunteer ranking, and other external services.

- **Data Transfer Objects (DTOs):** They play an important role in establishing the relationship between database objects and controllers, providing stability to the APIs even when

any changes occur in the database schema.

- **Middleware:** Crosscutting concerns like JWT authentication, authorization policies for roles, CORS policy configuration, and serving files have been implemented within the ASP.NET Core pipeline.

3.2.4 Design of Data Layer

Microsoft SQL Server is used as the database engine. The ORM tool for the project is Entity Framework Core [8, 9]. The following principles were used in designing the data layer:

- **Key Tables:** Users, Campaigns, Donations, Testimonials, VolunteerProfiles, VolunteerReports, Vouchers, ReserveFunds, and Withdrawals represent the core database tables.
- **Foreign Keys:** Proper delete rules (cascade, restrict, or set-null) are established on foreign key references to eliminate orphaned rows.
- **Monetary Types:** To avoid precision issues in monetary calculations, decimal(18,2) is used for all money columns.
- **Indexing:** Commonly searched columns like users' emails are indexed to improve search performance even with growing dataset size.

3.3 Development Process

Agile sprints were used in development process [28, 29]. Each sprint took about two weeks, after which there was a review of achievements followed by decision on priorities for the next steps. This helped to modify the plan when deficiencies became apparent during early testing stages.

3.3.1 Frontend Development

Individual interfaces have been developed for each individual:

- **Admin Interface:** Dashboard interface with options for managing users, accessing statistics, and monitoring the campaigns' progress.
- **Donor Interface:** Donation submission pages, testimonial submission page, and account page.
- **Volunteer Interface:** Progress tracking and activity logging page.

3.3.2 Backend Development

The backend was developed in modules, where each module had specific functions. The following list summarizes the purpose of each module:

- **User Authentication and Authorization:** Stateless JWT login, email confirmation link, second factor authentication using One Time Passwords (OTP), and role-based access control [23].
- **Donation Process Management:** Comprehensive donation process workflow from donation record creation to SSLCommerz payment integration, receipt generation, and real-time campaign total updates.
- **Campaign Management:** CRUD functionality alongside campaign progress tracking, milestones, campaign updates, and volunteer coordination.
- **In-Kind Donations Processing:** Entry of in-kind donations along with details about items and their value estimates, authentication of the donor using an SMS-based OTP, and integration into the accounting system.
- **Voucher System and Expense Management:** A workflow for vouchers, involving document upload and status updates throughout the approval process.
- **Fund Withdrawal and Reserve Management:** Proper request for withdrawals from the organization's account, public visibility of the reserve fund, and protection against any misappropriation of funds.

- **Sentiment Analysis:** Automatic categorization of comments based on whether they are positive, neutral, or negative, and subsequent mandatory admin intervention for any negative comment.
- **Artificial Intelligence:** Demand prediction for donations using Singular Spectrum Analysis, donor churn prediction, anomaly detection on donation streams, and prediction of success rates for campaigns.
- **Data Analytics:** Summary reports regarding donation statistics, campaigns, volunteer contributions, donor demographics, and other related information, which is accessible via API endpoints.
- **Automated Notifications:** Automated emails to notify the donors, SMS-based OTPs, campaign notifications, and automatic administrative notifications for different events.

3.4 Testing

In both cases of frontend and backend, automated test cases along with scenario testing were used to ensure proper functionality. The main aim was to check the validity of business logic, API responses, and to ensure that no bugs appeared during transitions between releases. The critical nature of money and personal data handling in a philanthropy-based application demanded a strict testing pipeline, which became mandatory at the stage of project development. The overall testing approach consisted in separating each part of the system for speedy verification but also included conducting end-to-end integration cycles to mimic real life usage scenarios. The use of continuous integration allowed detecting bugs and errors introduced during changes at early stages of development, ensuring smooth integration of newly added tracking functionality into the system core.

Table 3.2: Testing Matrix

Test Level	**Frameworks**	**Objective**
Frontend Unit/Component	Vitest, React Testing Library	Check individual components, Redux slices, and client-side service functions in isolation.
Frontend Contract	MSW	Confirm that API wrapper functions send the right requests and handle responses correctly, using mocked endpoints.
End-to-End (E2E)	Playwright	Run through complete browser workflows like logging in, browsing campaigns, and stepping through the donation checkout.
Backend Unit/Integration	xUnit, Moq	Exercise controller and service methods under both normal and error conditions.
Security Regression	Manual API tests	Probe authorization boundaries, try malformed inputs, and verify payment callback validation.

Test code lives in well-defined locations within the project:

- **Frontend tests:** Found under `frontend/src/components/__tests__`, `frontend/src/store/slices/__tests__`, and `frontend/src/services/__tests__`.
- **Backend tests:** Located in `backend/DonationManagementSystem.Tests`, covering both unit-level and integration-level scenarios.
- **Script runners:** NPM scripts group test execution into categories such as unit, contract, coverage, and E2E so they can be run selectively.

Before any release build, a lightweight PowerShell script runs backend and frontend checks locally to catch regressions early.

3.5 Deployment Strategy

Deployment and packaging of frontend and backend are kept separate, allowing flexibility when hosting the services together or separating them:

- **Frontend:** The compiled frontend code is built using Vite into an optimized static file bundle and can be hosted using any generic web server.
- **Backend:** The ASP.NET Core Web API is deployed as a standalone self-hosted service; runtime configurations are specified in `appsettings` JSON files and environment variables.
- **Database:** Database schema modifications are done using Entity Framework Core migrations; automatic startup verification ensures the database schema is up to date.
- **Environment Variables:** Credentials for payment gateways, JWT signing key, and notifications service configurations are injected as environment-specific configurations.

3.6 Security Measures

Security was never an afterthought. On the contrary, all kinds of controls were applied at each and every level following the OWASP recommendations [19] and standard security practices [18]. In view of how sensitive the financial operations and personal donor data can be processed by this system, we opted for a defense-in-depth approach in order to reduce the number of possible attack vectors to an absolute minimum. Thanks to a well-designed security hierarchy, even if a certain line of defense fails for some reason, other layers will come to the rescue. As for regular vulnerability scanning and dependency analysis, they became integral components of our deployment pipeline to ensure constant readiness of the system to repel any attacks. The highest priority has been placed on confidentiality of all personal data stored and transmitted over the Internet while the role-based access control model helped maintain institutional trust.

Table 3.3: Core Security Controls

Threat	**How It Is Mitigated**
Credential compromise	Passwords are hashed with BCrypt before storage; sessions rely on short-lived JWT tokens [24, 23].
Privilege escalation	Every protected endpoint checks the caller's role claims before executing [10].
Payment callback tampering	Callback payloads from SSLCommerz are validated server-side before any financial records are updated.

Security Layers Explained:

- **Authentication:** JWT authentication uses bearer tokens with customizable issuer, audience, and expiry time settings [23]. Session refreshes occur without requiring a re-authentication step.
- **Password Handling:** Passwords are stored using the BCrypt hashing algorithm; password reset tokens undergo validation based on password hash comparison [24].
- **Access Control:** Access is restricted via RBAC policies that determine which endpoints are accessible by certain roles so that admin-specific endpoints are out-of-bounds for both donors and volunteers [10].
- **Input Validation:** Incoming request inputs are validated server-side with DTO models, which helps mitigate any injection-based attacks and malformed data handling issues.
- **Network and Asset Security:** Cross-origin resource sharing is limited to known origins, while file uploads are hosted from controlled static file directories.

Chapter 4

System Architecture and Design

Chapter 4: System Architecture and Design

4.1 Overview

This chapter will discuss the architecture of the system, starting with the layers and going all the way down to the database tables. In contrast to the previous chapter, which concentrated on the process of building an information system and its methodology, the present chapter will focus on the actual architecture and data model.

4.2 Frontend Architecture

The client side is built around React's component model, with a few key technologies holding it together:

- React components handle all UI rendering and user interaction.
- A Redux store provides one centralized place for application-wide state.
- Tailwind CSS takes care of styling and responsive behavior.
- TypeScript adds static type checking, which helps catch mistakes before they reach the browser.

4.3 Backend Architecture

The server-side API is written in .NET Core and sticks to RESTful design principles [7]. Its internal structure keeps different responsibilities cleanly separated:

- **Controllers:** Accept HTTP requests, validate inputs, and return properly formatted responses. Each feature module has a dedicated controller.
- **Services:** Where the business rules actually live. Domain operations, cross-module coordination, and anything requiring non-trivial logic are handled at this layer.

- **Data Access:** Entity Framework Core, via the `AppDbContext` class, manages database queries and persistence.
- **Middleware:** Handles concerns that cut across the whole application: authentication checks, authorization enforcement, CORS policies, and serving uploaded files.
- **DTOs:** Request and response shapes are defined as separate data contracts, so internal schema changes do not automatically break the external API surface.

4.4 Database Schema and Data Dictionary

All persistent data architecture is modeled using an enterprise-quality relational database using Microsoft SQL Server. For ensuring proper management of the schema throughout its lifetime and implementing proper version control, Entity Framework Core controls all aspects of the schema using automatic code-first migrations [9]. The Object Relational Mapping process automatically provides a safeguard against any potential query injection vulnerabilities while supporting predictable evolution of the database schema along with the C# codebase. The data dictionaries shown below in the following subsections reflect some of the operational entities along with strict requirements of data types.

4.4.1 Table Campaigns

`TCampaigns` is an entity that forms the core of fundraising activities, from the inception through progress tracking to either successful achievement or failure to raise funds. It maintains financial targets with its fields, namely `TargetAmount` and `CollectedAmount`, which enable the calculation of progress percentages. The table also ensures auditing capabilities regarding the status of campaigns, clear deadlines, and foreign keys pointing at the actual admin creating such campaigns.

Table 4.1: Data Dictionary - Campaigns Entity

Column Name	Data Type	Constraint	Description
Id	INT	PRIMARY KEY	Auto-increment campaign identifier.
Title	NVARCHAR(200)	NOT NULL	Campaign title displayed to donors.
TargetAmount	DECIMAL(18,2)	NOT NULL	Goal numerical value.
AmountRaised	DECIMAL(18,2)	DEFAULT(0.0)	Cumulatively dynamically updated value.
Status	NVARCHAR(50)	NOT NULL	State such as pending, active, completed, cancelled.
CreatedBy	INT	FOREIGN KEY	User who created the campaign.
CreatedAt	DATETIME2	DEFAULT(GETDATE())	Record creation timestamp.

4.4.2 Table Donations

The `Donations` table serves as the main financial ledger for the system, carefully documenting every donation transaction, which also helps in forming the essential many-to-many relationship between donors on the platform and their corresponding charity campaigns. The inclusion of minute details in this table like the exact amount transacted, transaction time stamp, and type of payment method used allows complete transparency regarding all financial transactions. Such minute information is crucial for creating financial receipts for donations and auditing purposes, along with providing real-time analysis of the financial state of the system.

Table 4.2: Data Dictionary - Donations Entity

Column Name	**Data Type**	**Constraint**	**Description**
Id	INT	PRIMARY KEY	Donation record identifier.
CampaignId	INT	FOREIGN KEY	Parent campaign reference.
UserId	INT (nullable)	FOREIGN KEY	Registered donor reference (nullable for guest donations).
Amount	DECIMAL(18,2)	NOT NULL	Donation amount.
PaymentReference	NVARCHAR(MAX)	NULLABLE	Gateway transaction reference value.
Status	NVARCHAR(50)	NOT NULL	pending, completed, failed, or refunded.

4.4.3 Table Volunteer Reports

The volunteers themselves have the option to submit a report regarding misconduct, absence, or any other kind of infraction committed by other volunteers. The data about these reports and their review statuses will be stored in this database table. In order to keep track of the situation and ensure that there are no problems with the volunteer's work environment, each report submitted will be recorded meticulously with information about the perpetrator, the one who reported him or her, and the classification of the infraction. The administrative personnel will then check these outstanding entries, determine their validity, collect more evidence, if needed, and make the final decision whether to warn or suspend accounts.

Table 4.3: Data Dictionary - Volunteer Reports

Column Name	Data Type	Constraint	Description
Id	INT	PRIMARY KEY	Report identifier.
ReportedByVolunteerId	INT	FOREIGN KEY	Volunteer profile that submitted the report.
ReportedVolunteerId	INT	FOREIGN KEY	Volunteer profile being reported.
ReportType	NVARCHAR(MAX)	NOT NULL	Type such as misconduct, no-show, or other.
Description	NVARCHAR(MAX)	NOT NULL	Detailed narrative of the incident.
Status	NVARCHAR(MAX)	NOT NULL	pending, under_review, resolved, or rejected.

Beyond these three core tables, the database includes several other entities that round out the platform:

- **Users:** Stores identity data, assigned roles, and profile information.
- **Testimonials:** Holds donor feedback along with computed sentiment scores and moderation flags.
- **CampaignUpdates:** Contains progress posts and milestone announcements published by campaign creators.
- **PhysicalDonations:** Tracks in-kind contributions collected by volunteers, with OTP verification records.

- **Vouchers, ReserveFunds, Withdrawals:** Together, these tables manage expense approval workflows, reserve fund balances, and controlled disbursements.

4.5 ER Diagram

The ER diagram below maps out the relationships between entities with one-to-many links between Users and Donations, Campaigns and Donations, and so forth.

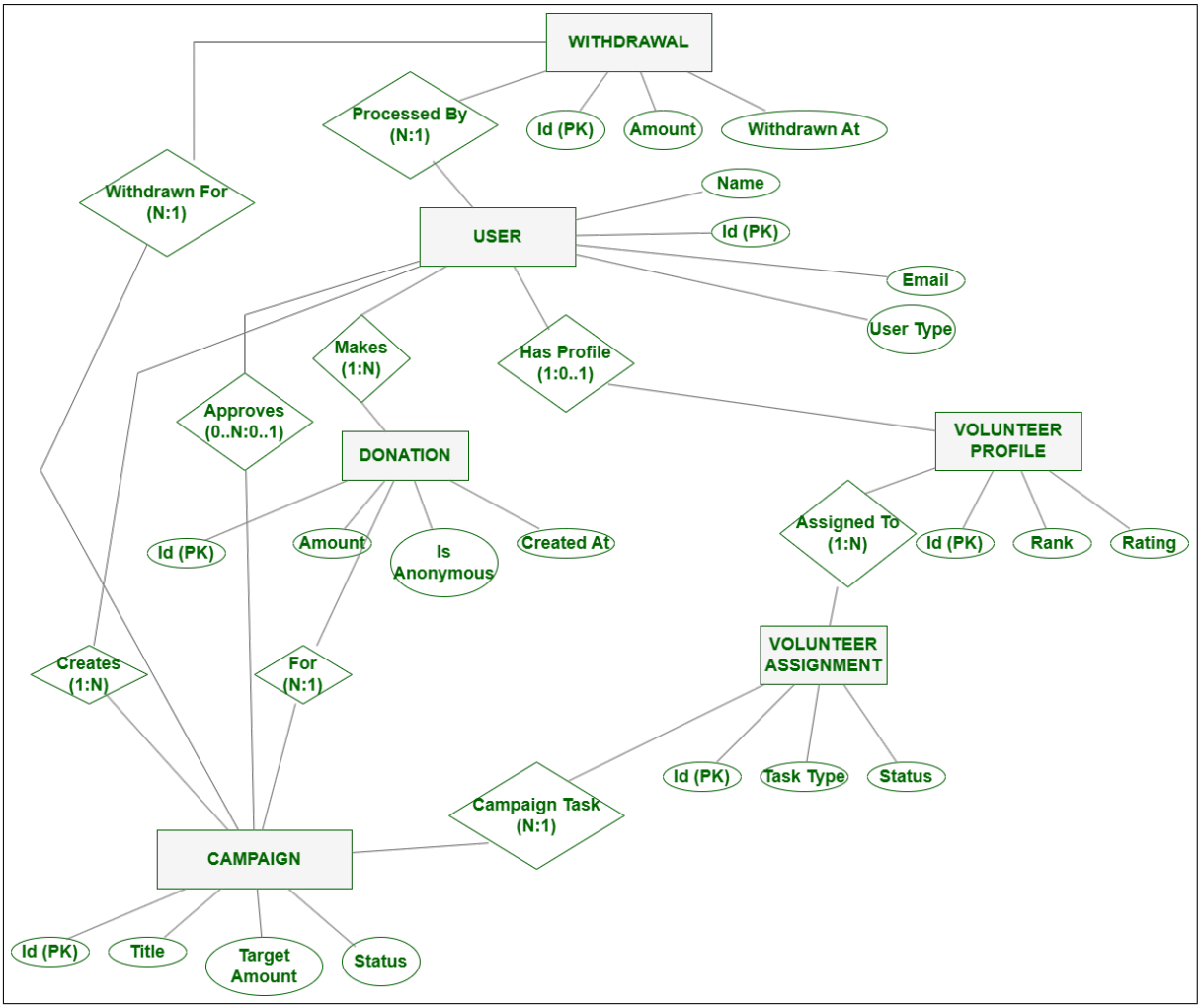

Figure 4.1: Entity-Relationship Diagram.

4.6 Use Case Diagram

The use case diagram below shows which actions are available to each role. Admins have the broadest access, while Donors, Volunteers, and Campaign Creators each see a focused subset of the system:

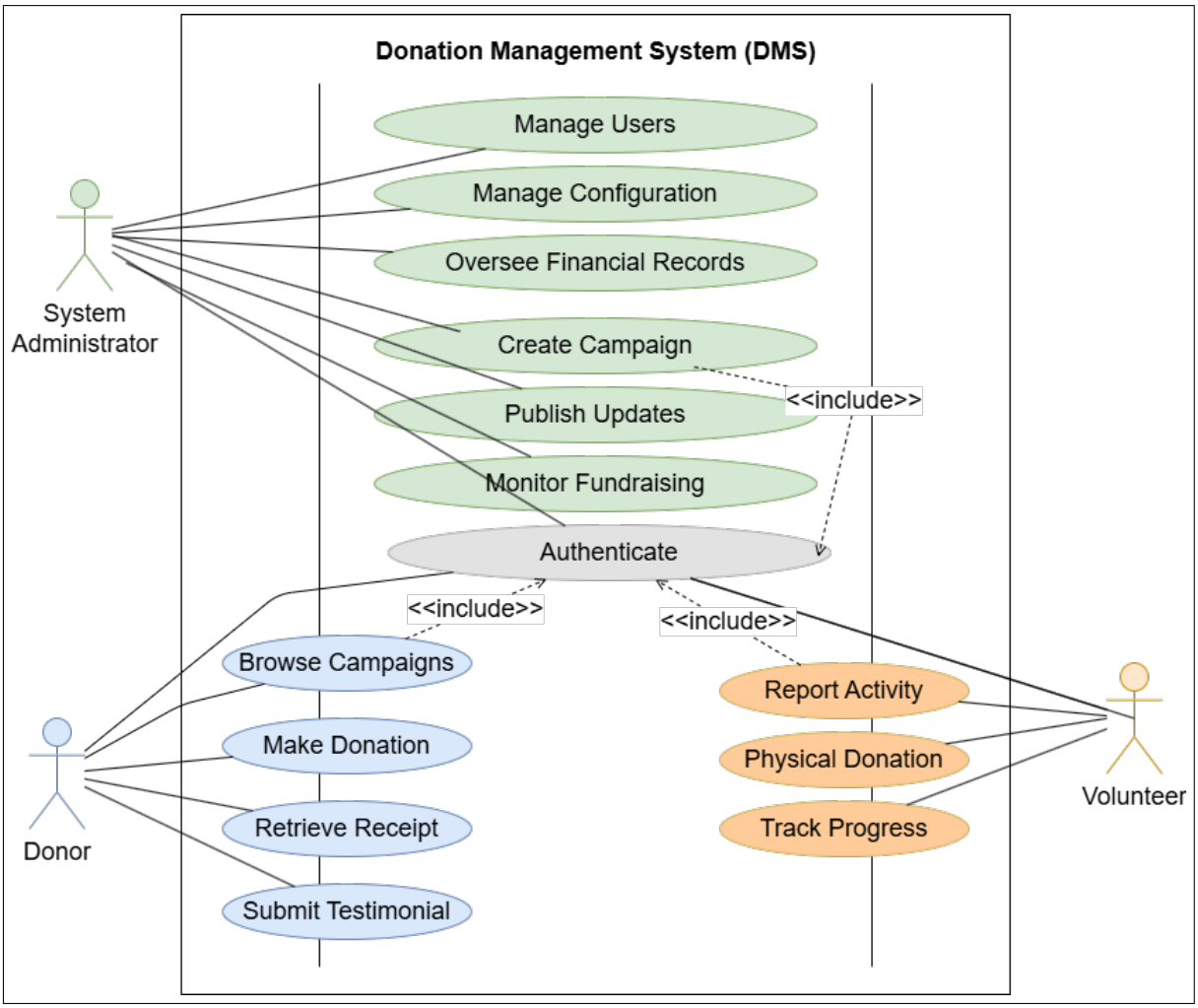

Figure 4.2: Use Case Diagram.

4.7 Schema Diagram

This diagram provides a closer look at every table in the database, including column names, data types, and the foreign key connections between them:

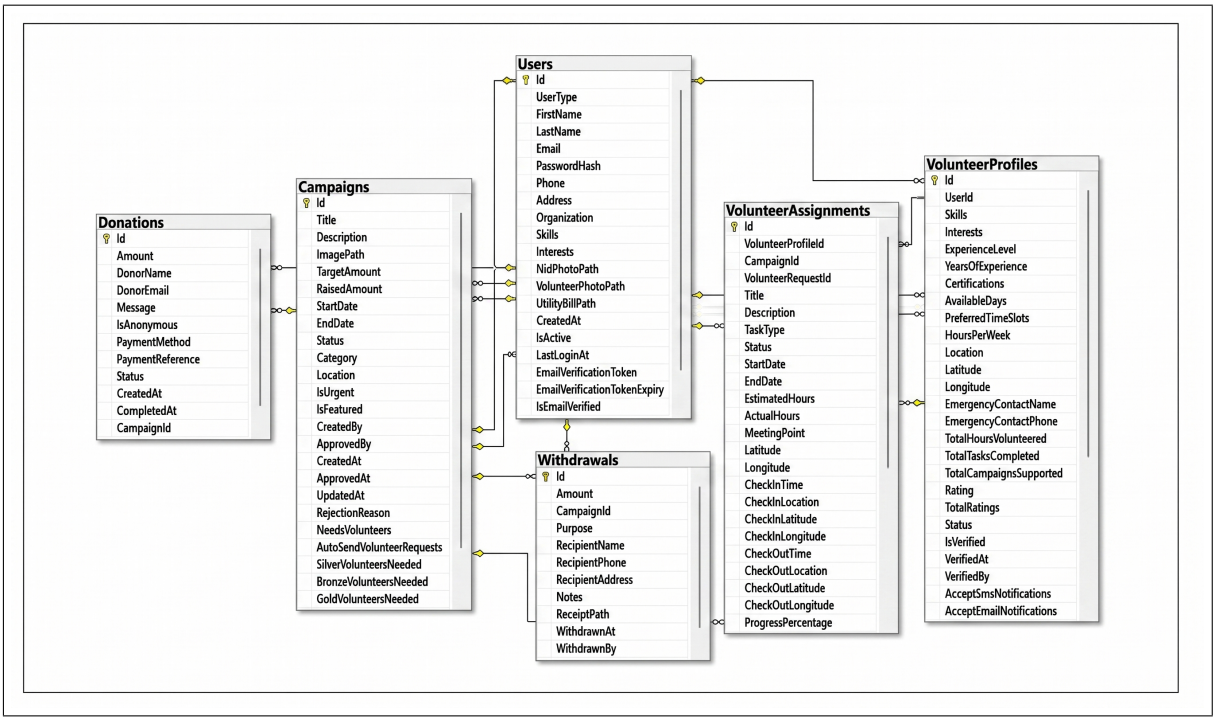

Figure 4.3: Database Schema Diagram.

4.8 System Context Diagram

At the highest level of the C4 model (Level 1), we can see the DMS interacting with external actors and external systems like SSLCommerz and email/SMS providers.

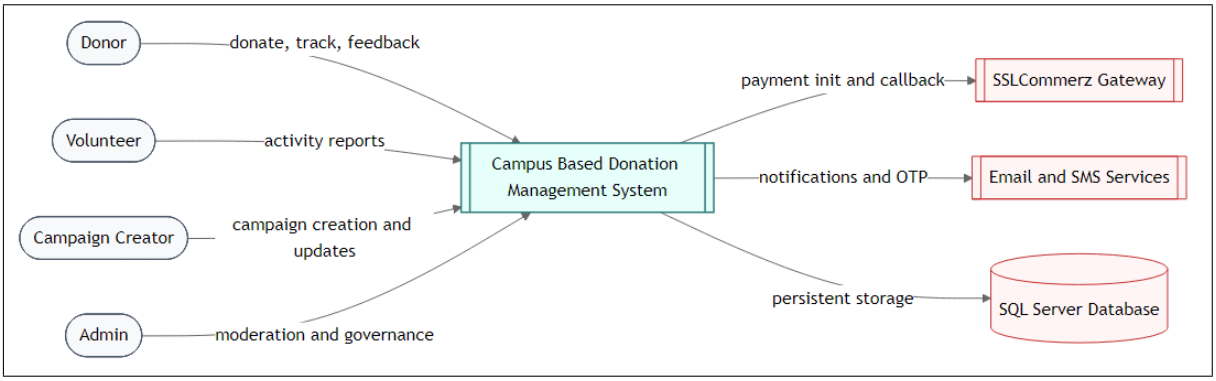

Figure 4.4: System Context Diagram.

4.9 Container Diagram

Going one level deeper (C4 Level 2), the container diagram breaks the system into its major runtime pieces: the React SPA, the .NET API, the SQL Server database, and the third-party services it depends on.

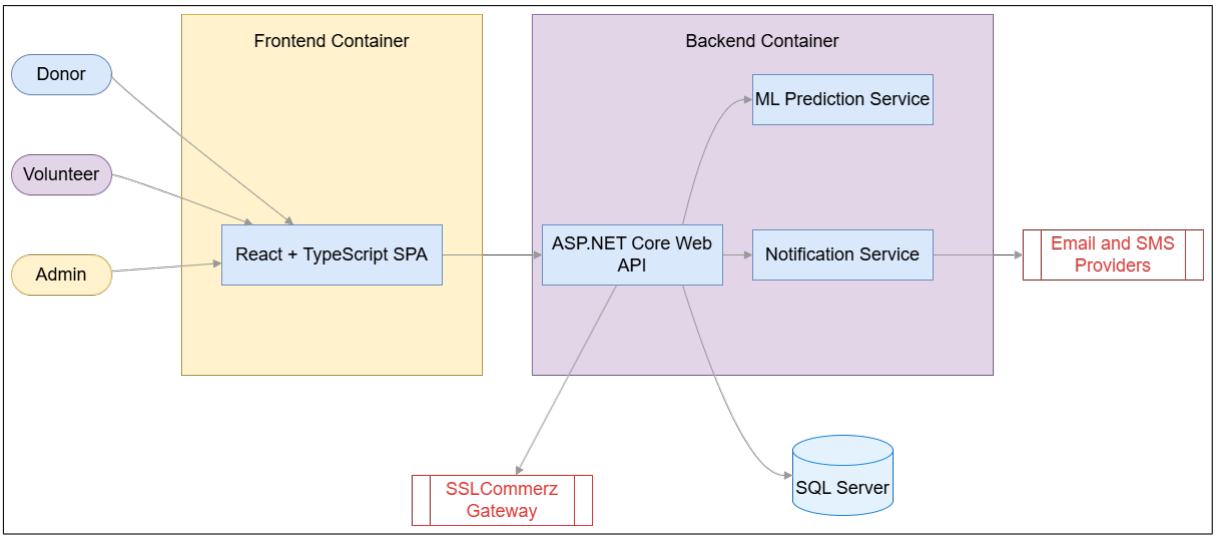

Figure 4.5: Container Diagram.

4.10 Component Diagram

Within the backend container, the component diagram depicts the relationships between controllers, services, and data access classes. The system follows the layered design pattern rigidly to separate the responsibilities of the components. The controllers serve as the first point of interaction for the incoming HTTP requests, passing the business logic and orchestration responsibility to dedicated service classes. The services handle the workflows, API integration, validation, and finally pass the operation to the data access layer. Data access classes communicate directly with the Entity Framework Core, ensuring all the database interactions are safe and parameterized. Decoupling of each component allows easy unit testing and scalability since changing any component would have minimal effect on other components.

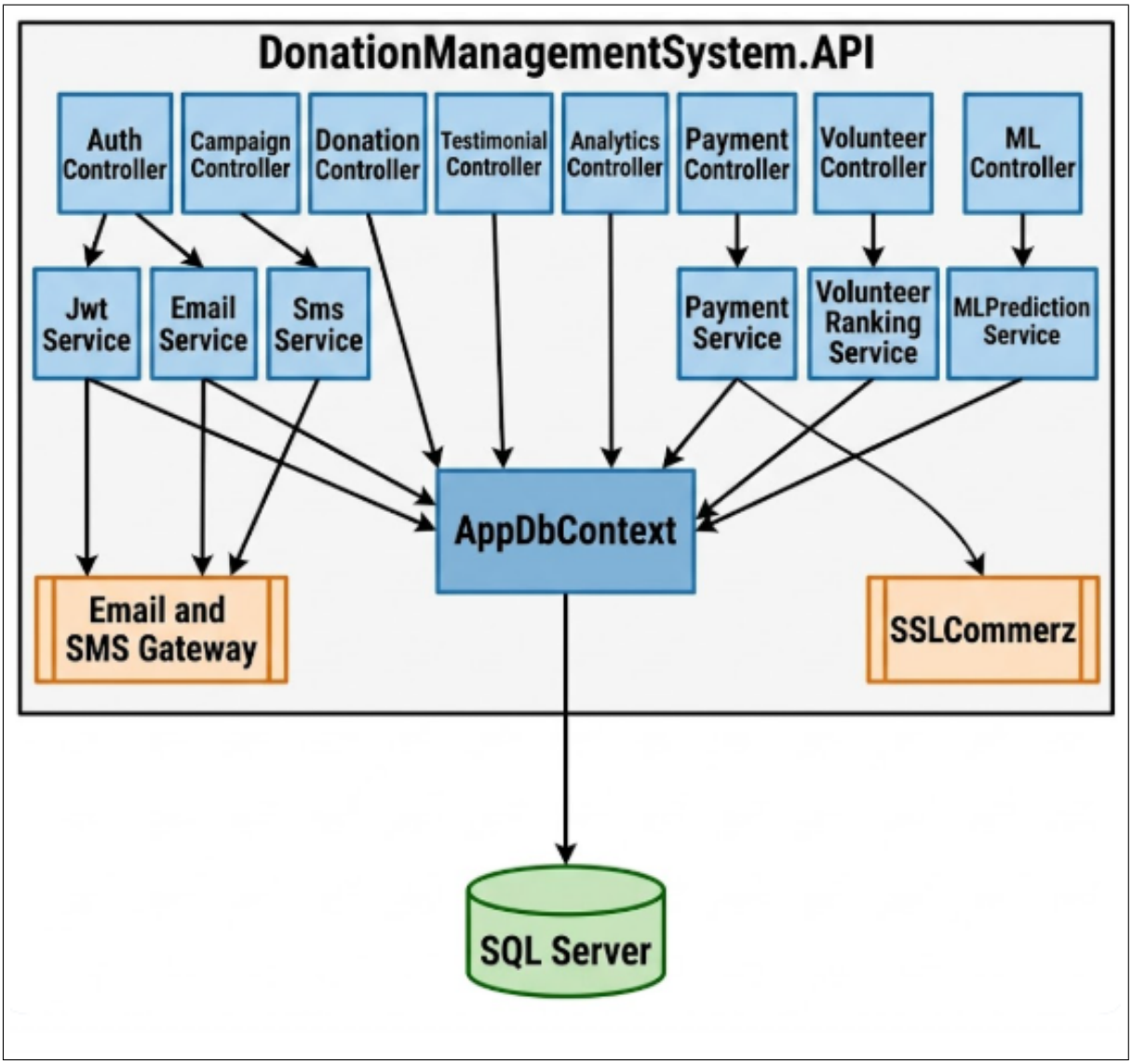

Figure 4.6: Component Diagram.

4.11 Class Diagram

A simplified class diagram captures the main domain objects such as Users, Campaigns, and Donations and the relationships between them.

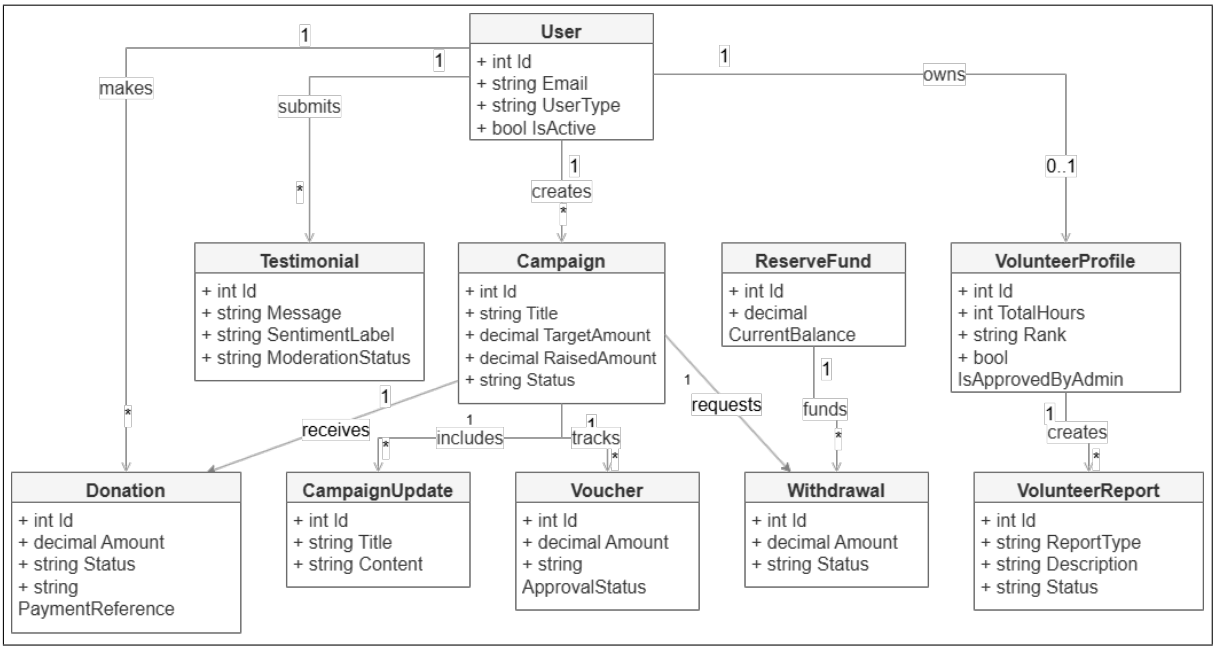

Figure 4.7: Class Diagram.

4.12 Data Flow Diagram (Level 0)

The Level 0 DFD draws a boundary around the entire system, showing external entities (donors, admins, payment gateway) and the high-level data stores they interact with.

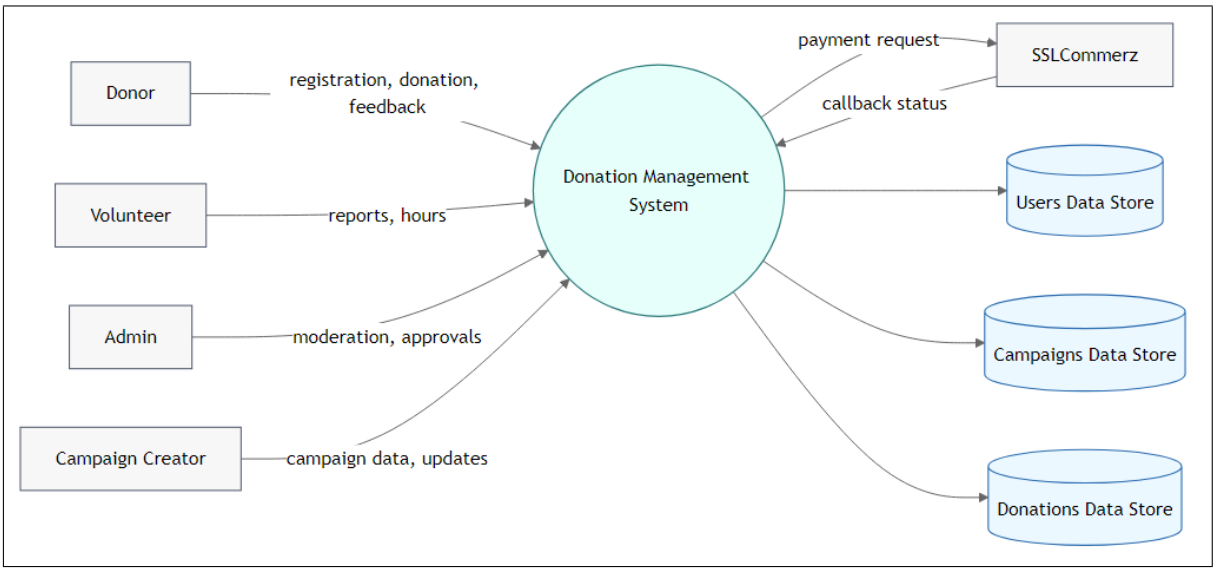

Figure 4.8: Data Flow Diagram (Level 0).

4.13 Data Flow Diagram (Level 1)

Drilling into Level 1, the major process is split into module-level groups like authentication, donations, campaigns, and analytics while the data movements between them become visible.

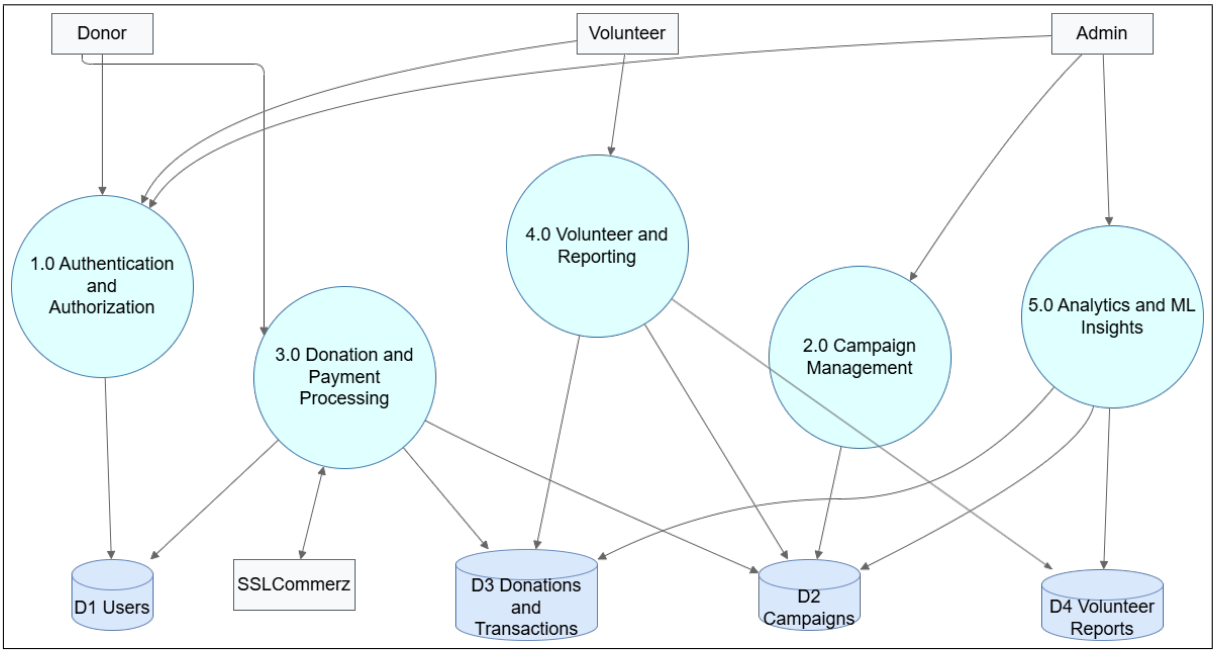

Figure 4.9: Data Flow Diagram (Level 1).

4.14 Deployment Diagram

Deployment Diagram The deployment diagram represents a direct association between software containers and infrastructure nodes, as well as connectivity with other systems. It clearly demonstrates the physical implementation of the React front-end on static hosting infrastructure and ASP.NET Core API on separate application servers. In addition, the deployment diagram emphasizes the existence of a safe boundary of communication between the central cloud database and other third-party systems like SSLCommerz payment gateway and automatic SMS sending applications. This diagram is vital in comprehending the operational, network topology restrictions, and scalability characteristics of the real-life production system.

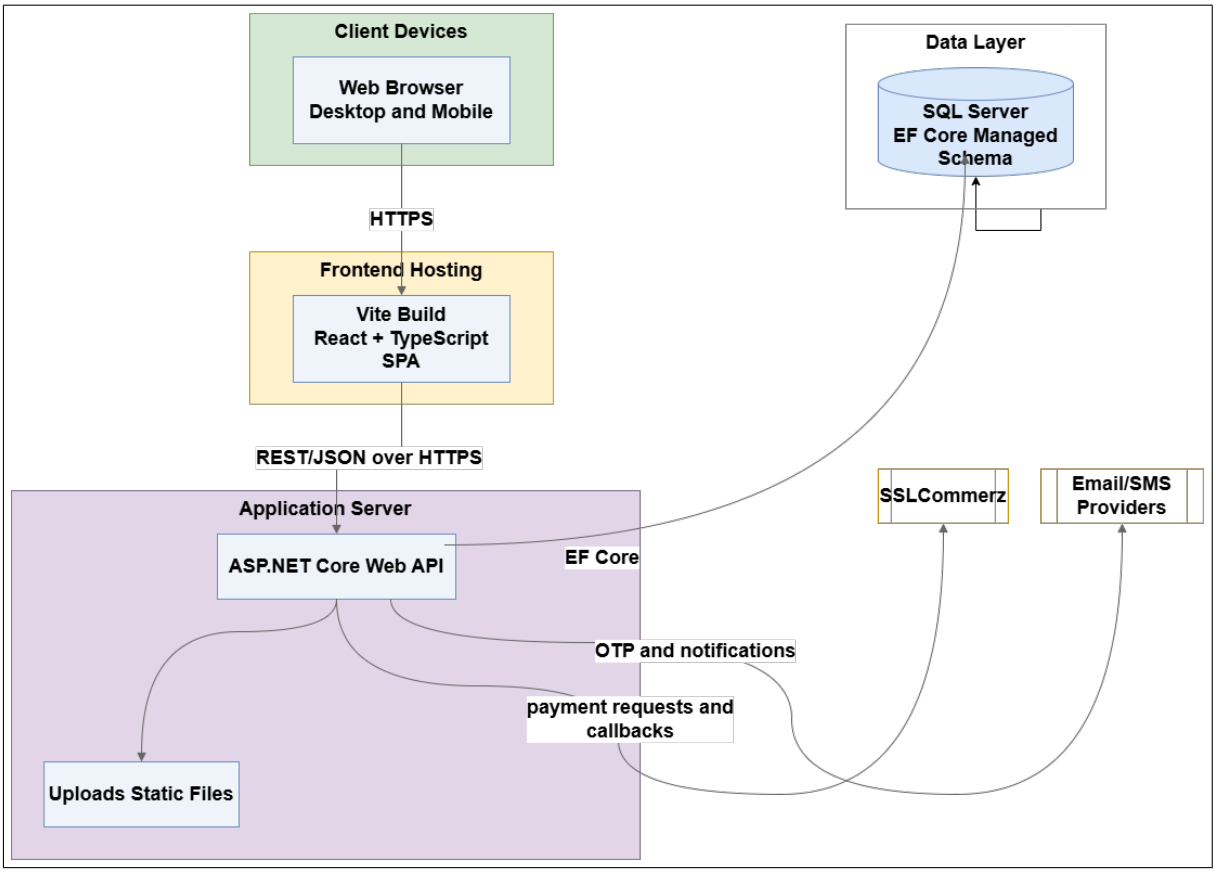

Figure 4.10: Deployment Diagram.

4.15 Activity Diagram: Donation Workflow

The process of executing these operations starts when a donor hits the “Donate” button and goes through the whole process until the end with a permanent update of the status of the donation operation in the database. The sequence of conditions to be considered for executing the flow has been identified in this diagram, for example checking whether the donor has been authenticated, generating the required payment payload based on the conditions met, and interpreting whether it was a success or failure in getting the response from the SSLCommerz gateway. With this diagram, one can easily understand how the process of data consistency has been maintained throughout the transaction cycle.

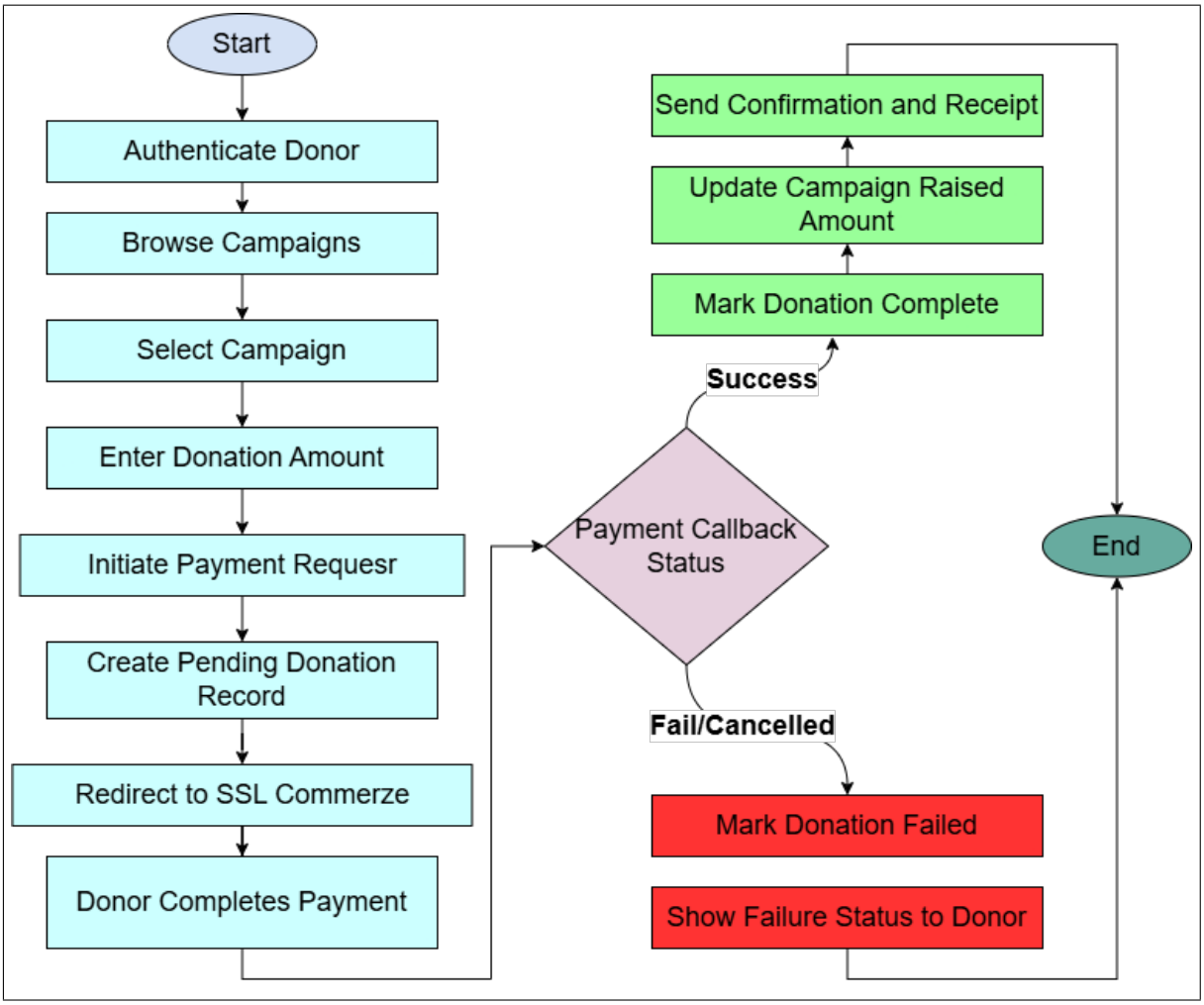

Figure 4.11: Activity Diagram for Donation Workflow.

4.16 Campaign Lifecycle State Diagram

Campaigns go through a series of states like Pending Approval, Active, Completed, and Canceled, but only some state transitions are permitted. The following diagram depicts these transitions. Logging of these state transitions and their authentication is mandatory to maintain accountability. For example, a campaign cannot be moved to an 'Active' state without any prior administration check. After a campaign attains a 'Completed' state, the process of fund settlement is initiated automatically.

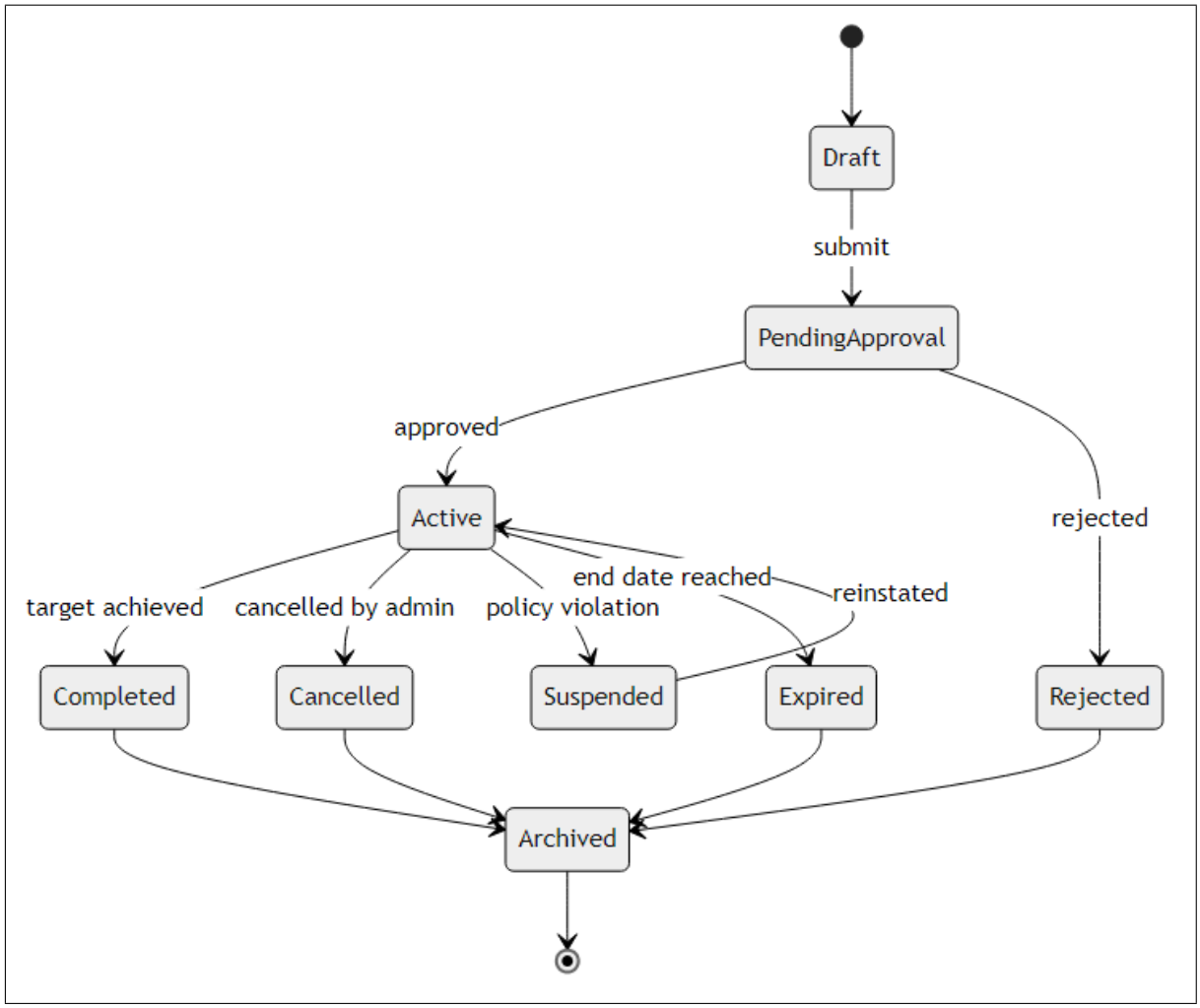

Figure 4.12: Campaign Lifecycle State Diagram.

4.17 Authentication Flow

Authentication relies on JWTs with role-aware authorization [23]. Here is how the process works in practice.

4.17.1 Registration, Login, and Token Issuance

1. The password for a new user gets encrypted using BCrypt before storage.

2. A token for email verification is created and emailed to the user's email address.

3. The server authenticates the user's credentials at the time of login and sends back an access token as well as a refresh token.

4. Upon expiration of the access token, the refresh token validates the token and generates a new access token without needing to authenticate the user again.

4.17.2 Authorization and Access Control

After authentication, all requests to secured endpoints receive further scrutiny:

- The JWT bearer middleware provided by ASP.NET Core verifies the integrity of the token's signature, issuer, audience, and expiration.
- Claims in the token are examined to ensure appropriate role constraints, meaning that only admin role tokens can access user management endpoints [10].
- Any actions related to profile modification need the identity claim in the token to be equal to that of the targeted user.

4.18 Payment Processing and Ledger Integration

Monetary transactions occur via the SSLCommerz payment gateway, and the post-payment sync is managed by the backend using callback notifications [3]. The gateway response is considered the ultimate authority for determining payment status, with ledger entries updated only when a valid callback notification occurs. Each individual transaction gets entered in the immutable ledger entry form for auditing purposes. A light retry mechanism is used to mitigate any temporary failure of callbacks.

4.18.1 Sequence Diagram: Payment Callback Flow

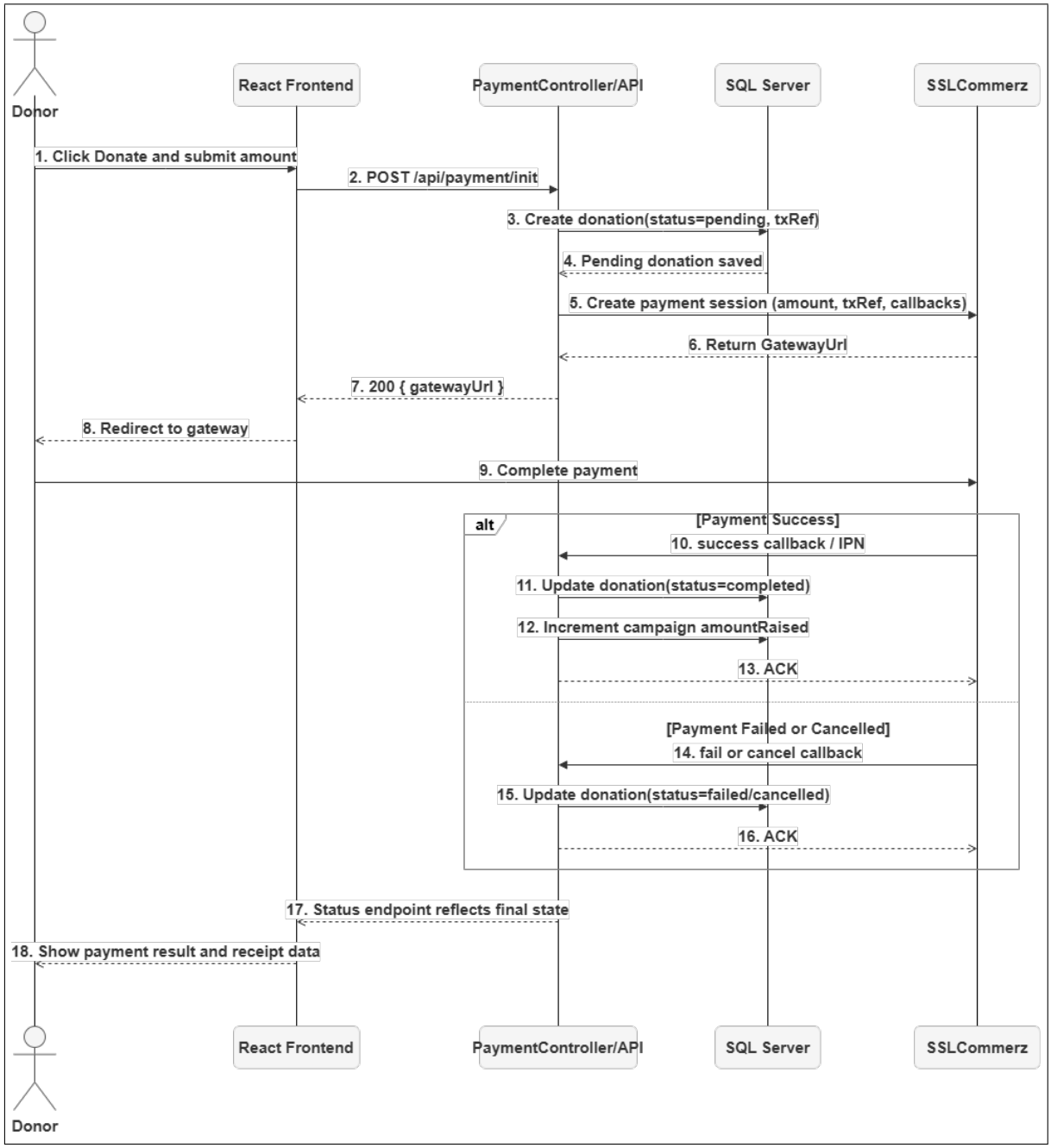

Figure 4.13: Sequence Diagram for Donation Payment Flow.

4.18.2 How a Transaction Starts

1. The donor hits the pay button on the frontend, triggering a server request.

2. The class `PaymentController` creates a record for the donation with a status "pending," along with an ID.

3. Data about the payment such as amount, donor information, and callback URLs is sent to the SSLCommerz API.

4. The browser of the user is then directed to the gateway URL provided by SSLCommerz.

4.18.3 Callback Handling and Idempotency

After completing the process, or failing to pay, the payment is communicated back to the server by SSLCommerz:

- The payload sent by the callback is mapped to the corresponding donation based on the donation ID or transaction reference.
- In case of a successful payment, the donation status changes to "completed," changing from "pending" previously.
- The campaign total is updated, while anything above the target is allocated to the reserve fund row.
- To avoid any duplicate balance updates, idempotency check is performed for the callback.

Chapter 5

System Implementation

Chapter 5: System Implementation

5.1 System Setup

The operation of the system involves two main components: React-based frontend that runs statically, and backend written in ASP.NET Core providing an API as well as a connection to the database. The current section covers utilized technologies and implementation of all functionalities through screenshots. Environment variables are used to separate environment-specific parameters like database connection strings and payment gateway keys for the development and production environments. The backend application makes use of appsettings files and secrets manager to get access to environment variables without any hardcoding of credentials. Entity Framework migrations help create the initial structure of the database, as well as reflect all changes in the source codebase. The frontend assets are built using Vite and may be served along with the API itself or any web server. Some scripts automate the process of starting both API and frontend in the local environment. Specific endpoints are available for making health check calls verifying the connection to the database.

5.1.1 Technologies Used

- **Frontend Technologies:**

Table 5.1: Frontend Technologies.

SN	Technology
1	React
2	TypeScript
3	Redux Toolkit
4	Tailwind CSS
5	Axios (HTTP Client)
6	Vite (Build Tool)

- **Backend Technologies:**

Table 5.2: Backend Technologies.

SN	Technology
1	.NET Core 7/8
2	C#
3	Entity Framework Core
4	SQL Server
5	ASP.NET Core Identity
6	JWT Authentication

5.2 System Implementation

The screenshots that follow are in order exactly as they would be seen by a user: signing in, using the admin control panel, setting up campaigns, donating, and so forth. This will help in understanding how various aspects of the system interact in practice.

5.2.1 User Authentication

The login page is the starting point. What happens behind the scenes? Sessions using JWT technology provide authentication, emails confirm account validity, and refresh tokens ensure continuous user logins without the need for passwords each time. An invalid login attempt is subject to rate limiting. BCrypt is used to hash passwords that are never saved in plain text. Logging out deactivates all refresh tokens.

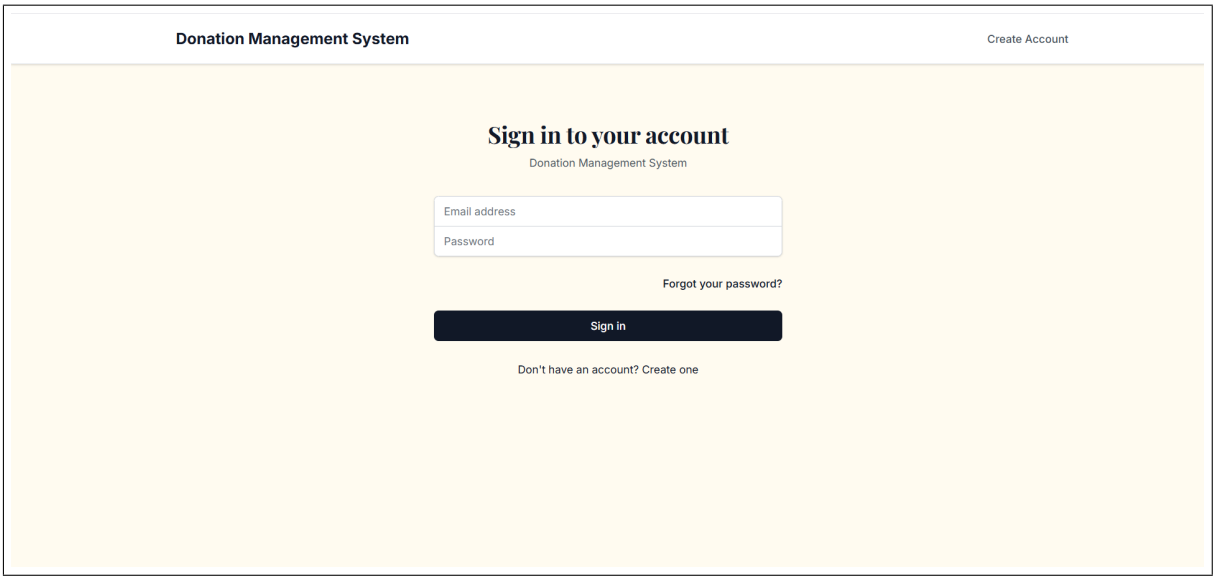

Figure 5.1: Login and Authentication Interface.

5.2.2 Dashboard Overview

Once logged into the system, a quick overview is available on the dashboard, which contains all the information that needs to be seen at once. This includes the total funds received, current active campaigns, pending reviews, and volunteer work.

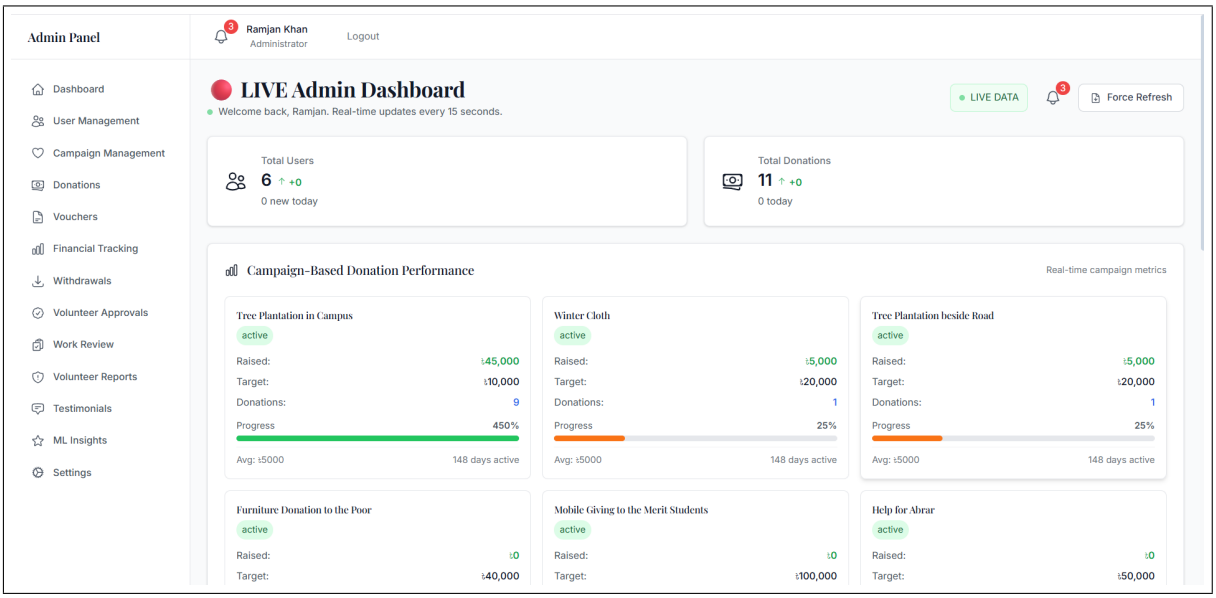

Figure 5.2: Admin Dashboard Overview.

5.2.3 User and Role Management

Admins can assign roles, enable or disable accounts, and manage user profiles from a dedicated panel.

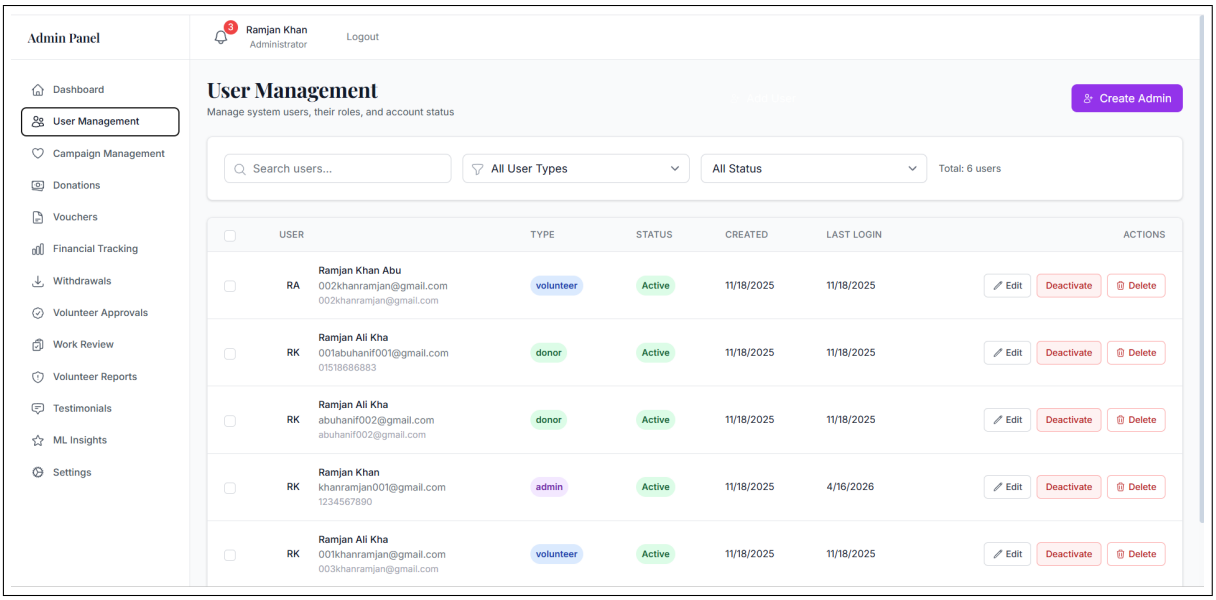

Figure 5.3: User and Role Management Panel.

5.2.4 Campaign Management

The creator of the campaign uses this interface to create new campaigns, set the amount needed, upload pictures, give updates on the progress and see how successful they have been in achieving their goals. For easy categorisation in listings and search queries, a campaign needs a name, description, category, and target amount. The start and end date of the campaign controls when it is exposed and prevents older campaigns from accepting donations.

Media files are checked for size and types to avoid slow loading times and improper file types. We time-stamp campaigns for accountability and update them. Administrators pause or archive campaigns if they break the rules or if they achieved their purpose. Campaign status indicators to inform the user of the current status of a campaign.

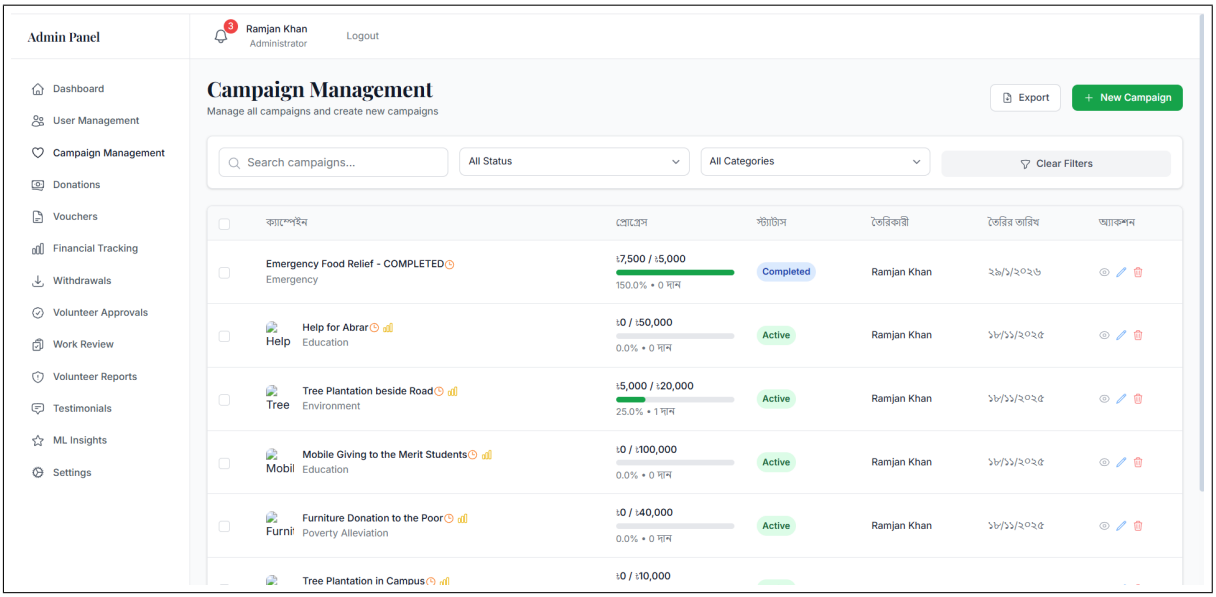

Figure 5.4: Campaign Creation and Management Interface.

5.2.5 Donation Processing

Donors go through a checkout flow that connects to SSLCommerz for payment. Once the transaction is confirmed, the system updates the donation status and campaign totals automatically.

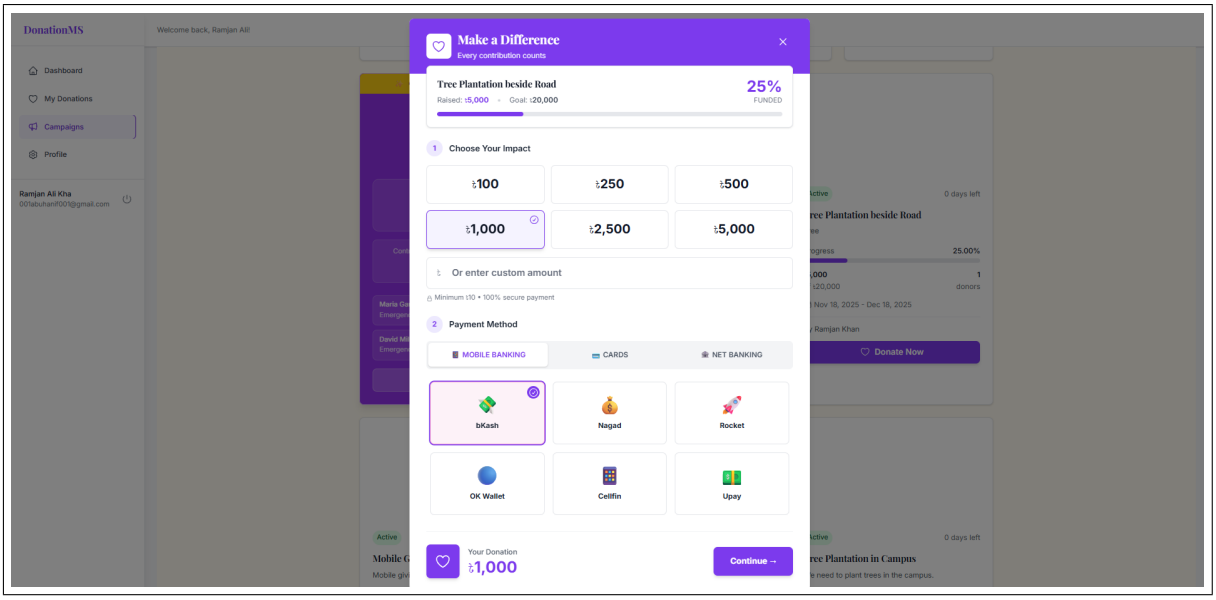

Figure 5.5: Donation Checkout and Payment Initiation.

5.2.6 Volunteer Management

Volunteers have their own panel where they can submit activity reports, log hours, see their overall progress, and check how their contributions stack up.

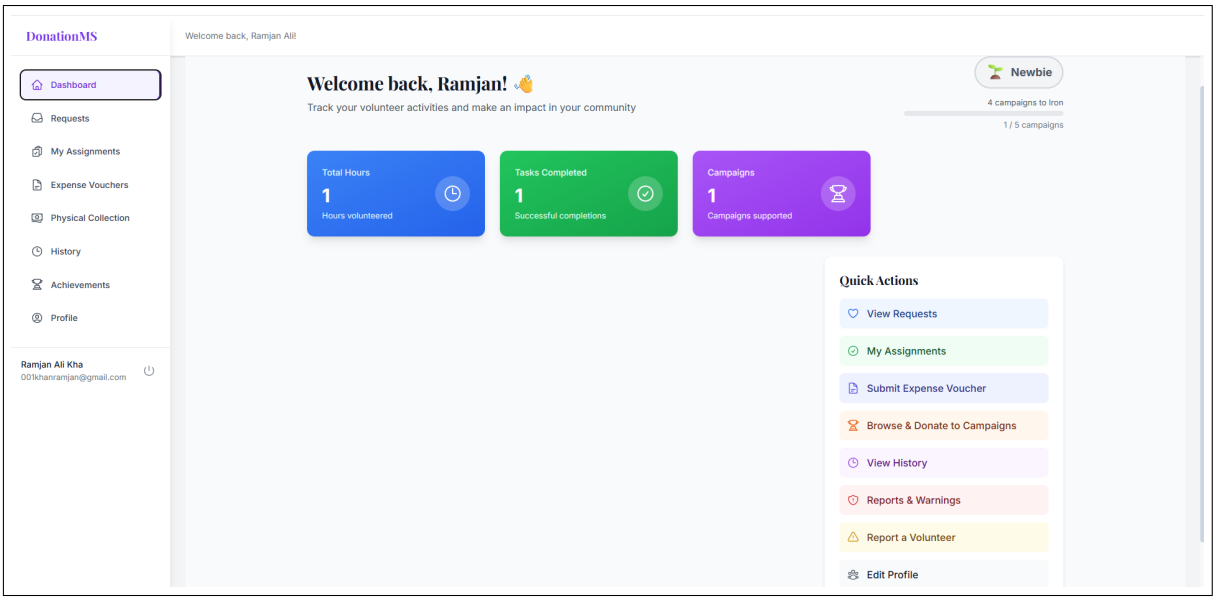

Figure 5.6: Volunteer Activity and Reporting Interface.

5.2.7 Testimonial and Sentiment Analysis

Donors who post testimonials are graded based on sentiment by the system. Any posts with negative sentiments are reviewed by the administration team before being displayed to the public. The sentiment analysis process is performed through machine learning models, which enable the system to determine whether the submitted content is of a positive, neutral, or negative nature. In doing so, the platform ensures that all donor testimonials displayed on its public forum do not contain any forms of inappropriate feedback. The moderation dashboard provides an interface where administrators have the opportunity to approve any comments that might offer constructive criticism but still be tagged as negative.

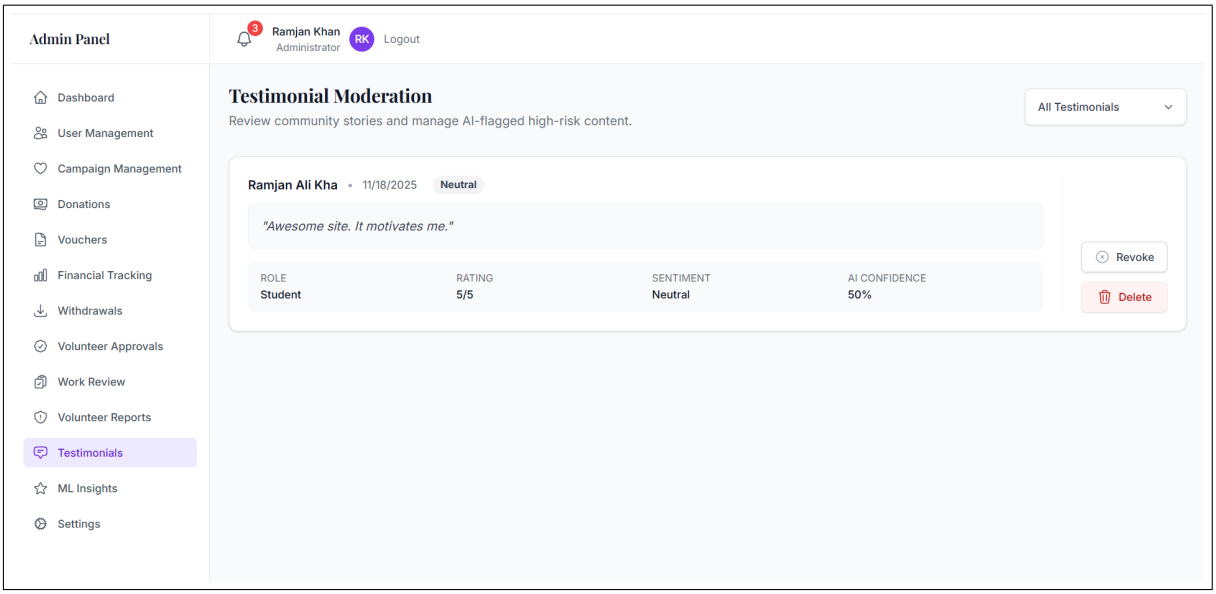

Figure 5.7: Testimonial Moderation and Sentiment Review.

5.2.8 Analytics and Reporting

The analytics dashboard pulls together donation trends, campaign performance, volunteer output, and donor activity into interactive charts and summary cards.

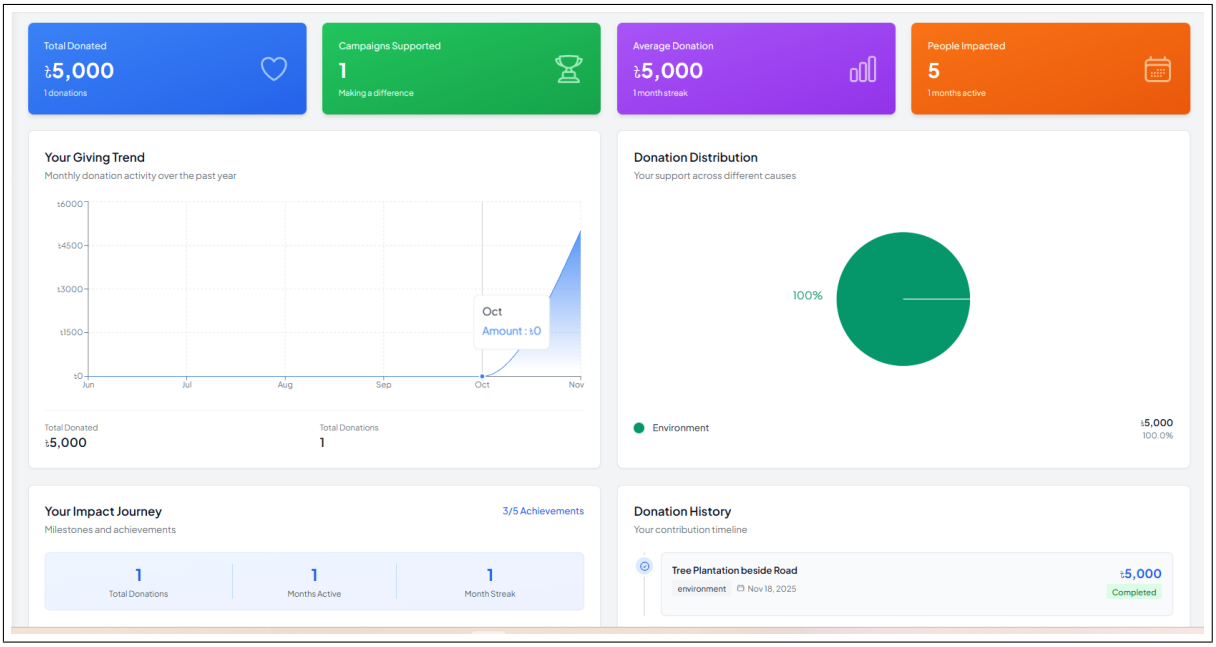

Figure 5.8: Analytics and Reporting Dashboard.

5.2.9 Additional Feature Screenshots

The remaining screenshots cover the other major modules built into the platform:

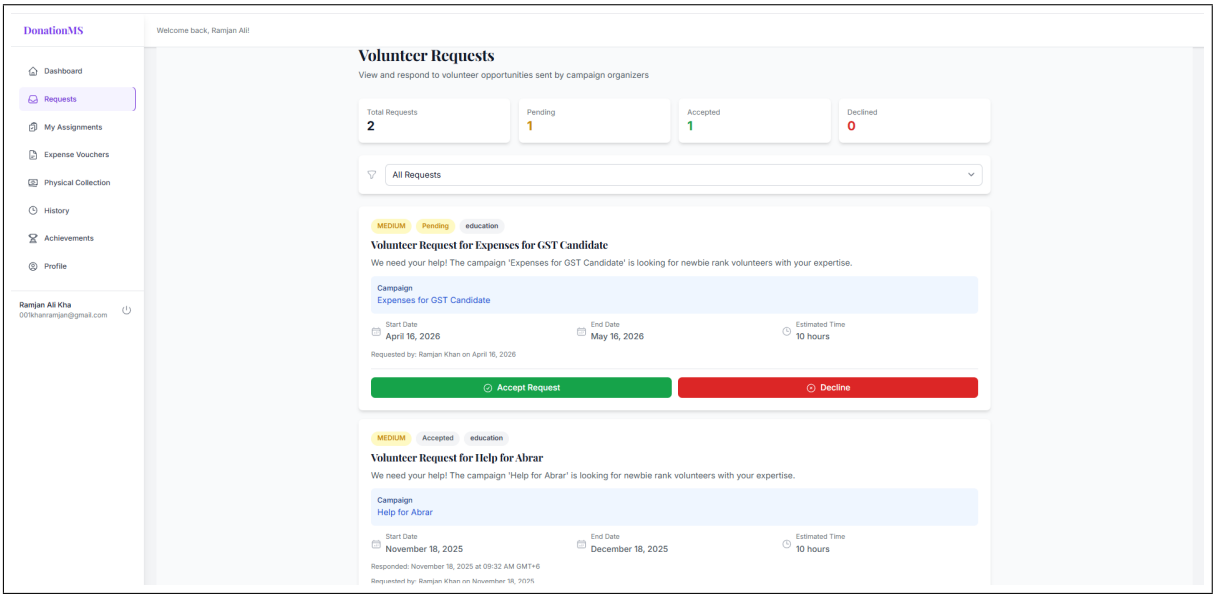

Figure 5.9: Campaign Updates and Announcement Interface.

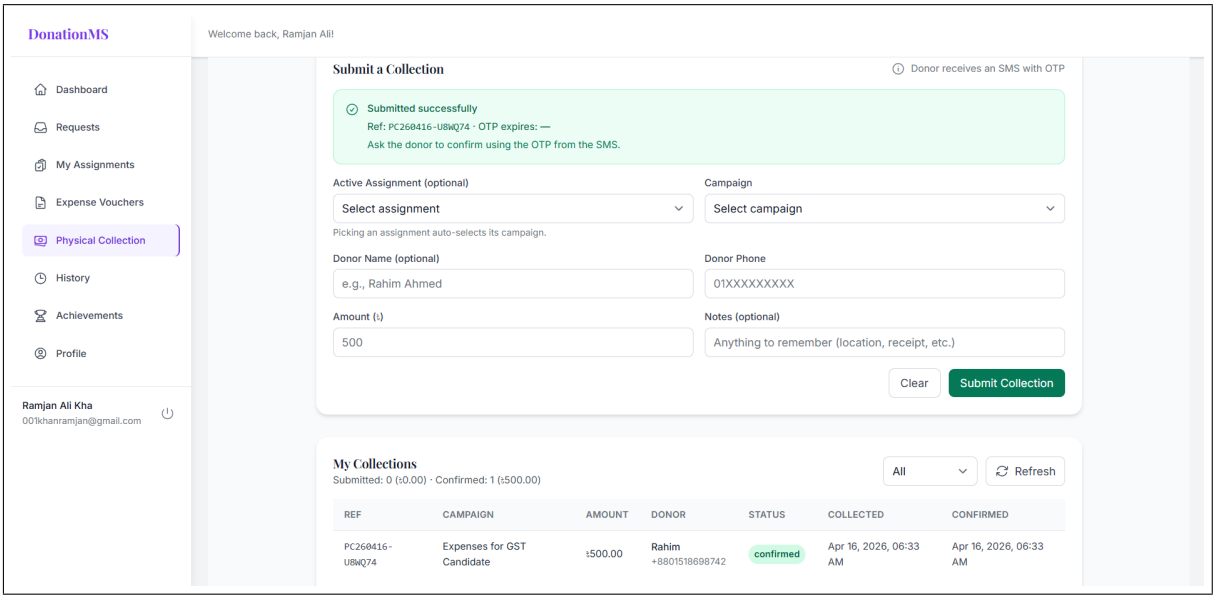

Figure 5.10: Physical Donation Verification and Tracking.

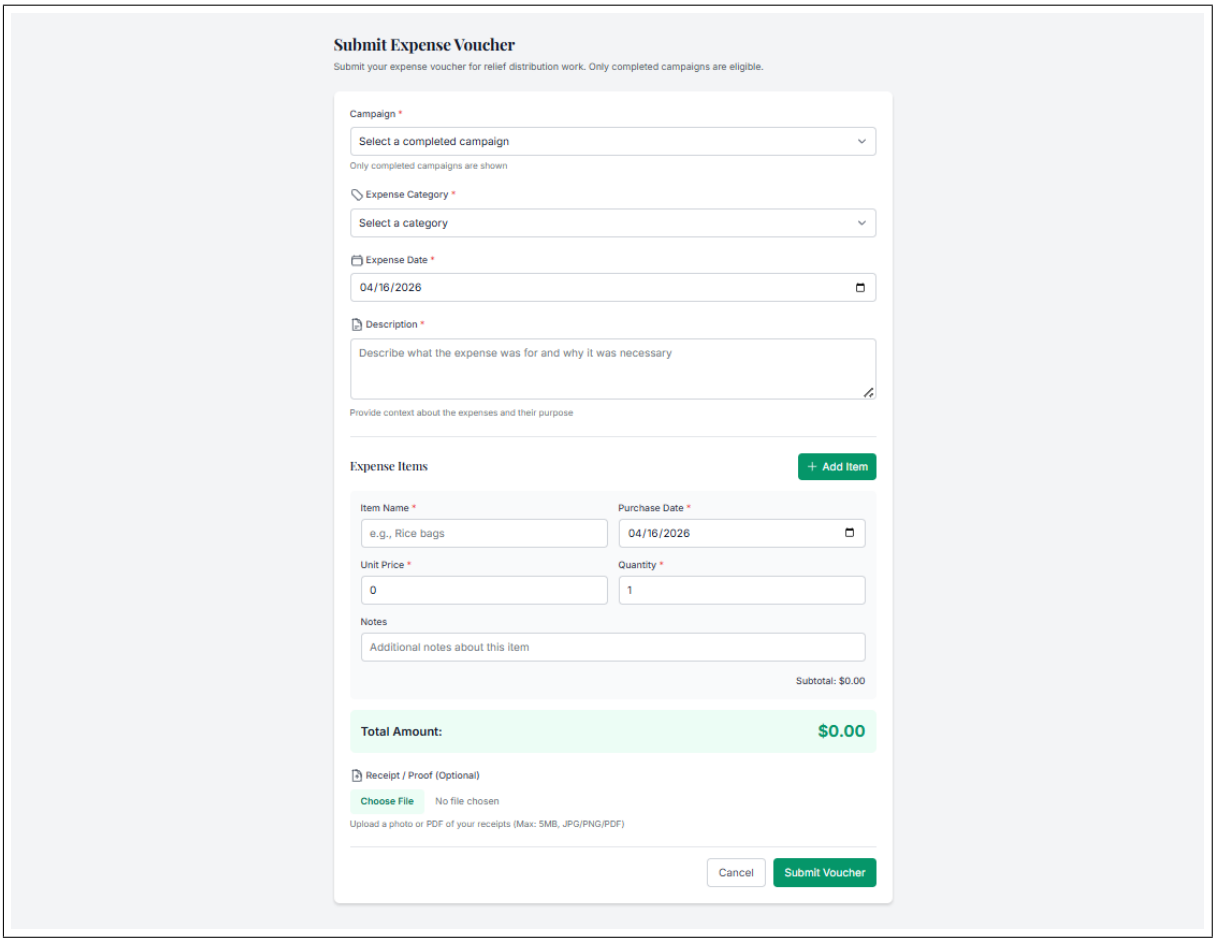

Figure 5.11: Expense Voucher Management and Approval Workflow.

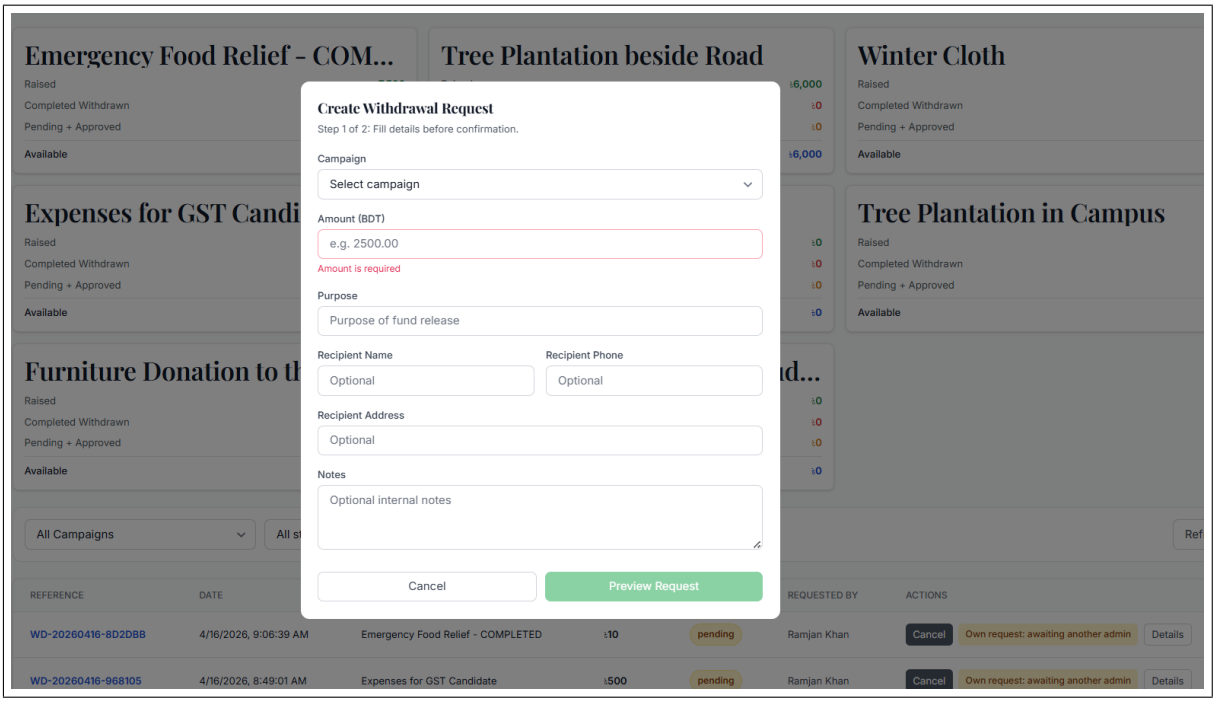

Figure 5.12: Withdrawal Request and Approval Interface.

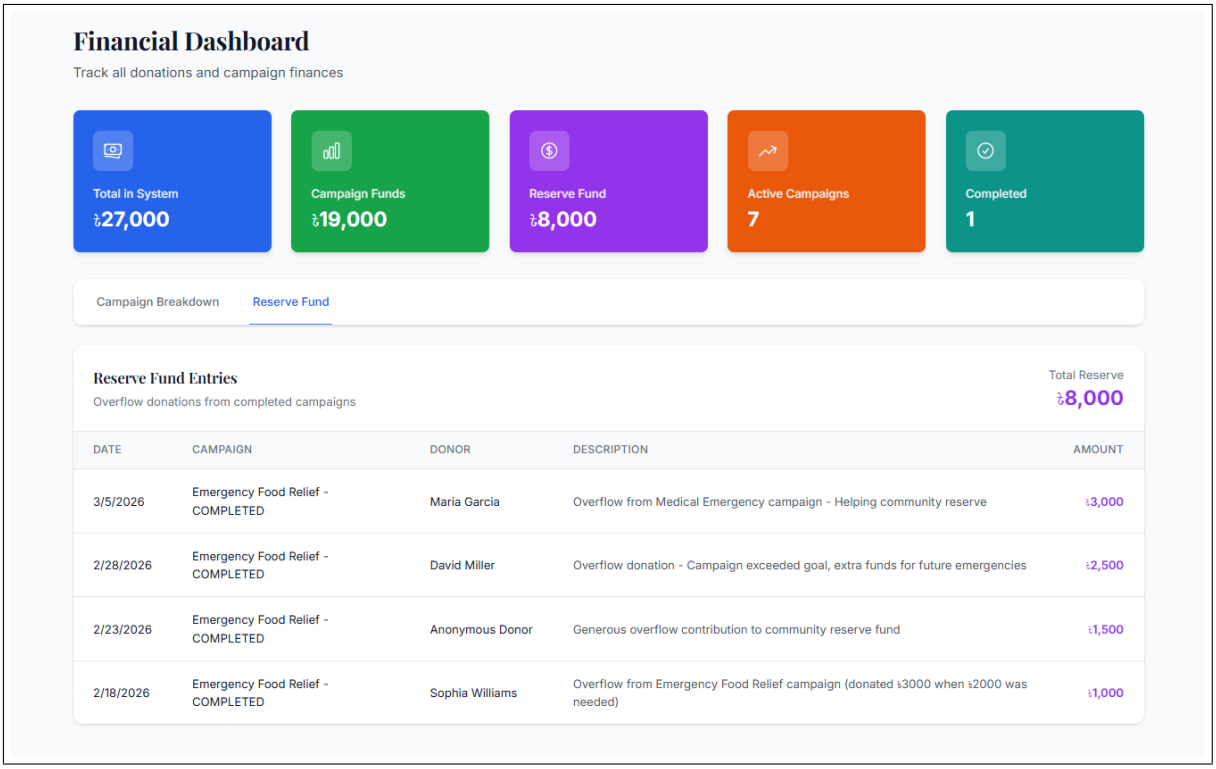

Figure 5.13: Reserve Fund Transparency.

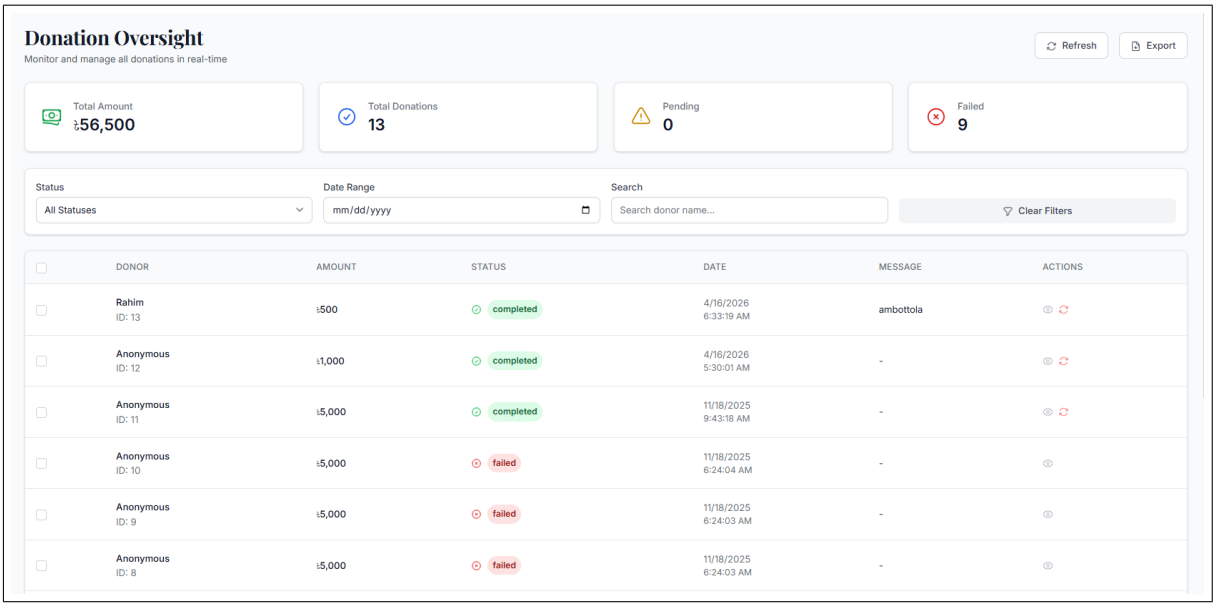

Figure 5.14: Payment Transaction History and Status Tracking.

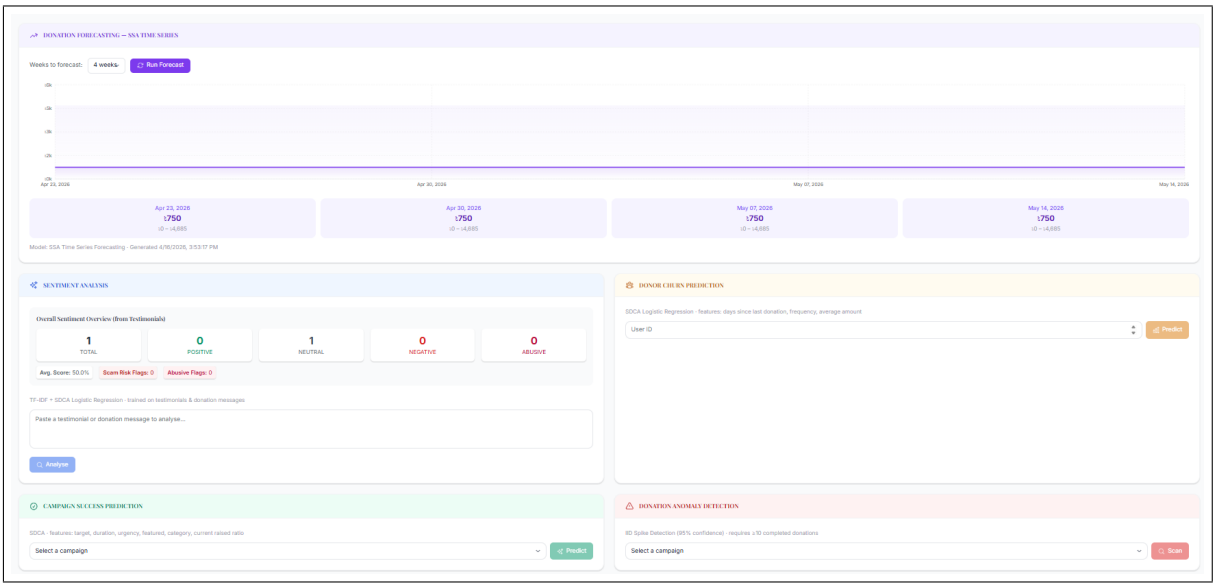

Figure 5.15: Machine Learning Insights and Prediction Outputs.

5.2.10 Screenshot File Arrangement

For consistency and maintainability, implementation screenshots should be stored under `Report/images` using the following filenames:

Table 5.3: Implementation Screenshot Naming Scheme

Filename	**Purpose**
login_page.png	Authentication/login screen.
admin_dashboard.png	Main administrative overview and metrics.
user_management.png	User, role, and access control management.
campaign_management.png	Campaign create/update and progress monitoring.
donation_checkout.png	Donation amount entry and payment initiation flow.
volunteer_panel.png	Volunteer reporting and activity tracking.
testimonial_moderation.png	Sentiment-based moderation and feedback review.
analytics_dashboard.png	Reports and analytical visualizations.
campaign_updates.png	Campaign announcement and progress update page.
physical_donation.png	Physical donation collection and OTP verification workflow.
voucher_management.png	Expense voucher submission and approval panels.
withdrawal_management.png	Withdrawal request queue and administrative decisions.
reserve_fund_dashboard.png	Public reserve fund summary and transaction visibility.
payment_transactions.png	Payment status log, callback result, and transaction history.
ml_predictions.png	Forecasting, anomaly, and churn prediction outputs.

5.3 Key Features Implementation

5.3.1 Donation Tracking and Processing

The donors and campaigners will receive real-time statistics on the amount of money collected for the goal. Every donation is stamped with the time, the amount of money donated and whether or not the transaction was successful. The module will be integrated with SSLCommerz for automatic generation of receipts and checking transactions.

5.3.2 Campaign Updates and Communications

Campaign creators post progress reports like stories about how donations are being used, achievements or even messages of gratitude. This keeps donors involved even after they've made their donations.

5.3.3 Volunteer Reporting and Recognition

The volunteers record their contributions through submission of reports detailing their hours of contribution, the nature of the contribution, campaigns they supported, and any outstanding achievement during the process. This information is used to rank the volunteers based on merits.

5.3.4 Physical Donation Management

In-kind contributions such as clothing, food, and materials are managed by the tracking module as follows:

- OTP based authentication of donors using SMS.
- Documentation and evaluation of contributions.
- Linking to the donation ledger system.
- Creation of audit trails.

5.3.5 Expense Voucher Management

Vouchers are used for campaign-related expenses in the following way:

- Structured vouchers, with documentation of all expenses.
- Approval hierarchies.
- Attachment of supporting documents.
- Status updates in real time.

5.3.6 Control on Finance and Withdrawals

No finance is withdrawn from the system unaccounted for:

- Withdrawal request procedures.
- Authorization processes.
- Dashboard showing the status of public reserve funds.
- Notification mechanisms.

5.3.7 Payment Integration

SSLCommerz Payment Gateway offers support for credit card payments, mobile banking (bKash, Nagad), and digital wallets. Transactions are automatically confirmed by server callbacks; thus, nothing is required from the donor or administrator after the transaction.

5.3.8 Machine Learning Features

Every functionality in ML is hosted within the backend as an in-process `MLPredictionService` powered by ML.NET framework. It is registered as a singleton service; hence, once the model is trained, it stays in memory, and there is no need for training each time.

5.3.8.1 1. Donation Demand Forecasting (SSA)

Weekly donations are input into the SSA pipeline from ML.NET for forecasting along with their confidence intervals [26, 30]. This allows the campaign managers to estimate the income they can expect during the upcoming weeks.

5.3.8.2 2. Donor Attrition Prediction

To classify potential donor attrition cases, the solution uses a binary classifier that includes variables such as the recency of the last gift, number of gifts, gift averages, and frequencies [27]. It outputs a risk score along with an interpretable risk level (low, medium, high).

The probability estimate is interpreted as:

$$P(Attrition) = \sigma\left(\sum_{i=1}^{n} w_i f_i\right) \tag{5.1}$$

where σ denotes the sigmoid mapping to a value in $[0, 1]$.

5.3.8.3 3. Donation Anomaly Detection

The IID Spike Detection technique implemented in ML.NET identifies any anomalies in the finished donation data streams. Examples of what may be detected by this technique are:

- Donations which are unusually large (small) compared to the campaign's usual donations;
- Periods of unusual high activity inconsistent with the behavior of the campaign.

Any identified anomalies will appear in the admin dashboard.

5.3.8.4 4. Sentiment Analysis

The testimonial text gets classified into sentiments of positive, neutral, or negative by means of logistic regression using SDCA classifier algorithm. Any testimonial that gets classified into the negative class is automatically flagged for moderation purposes [20].

5.3.8.5 5. Campaign and Volunteer Intelligence

The software can also predict the likelihood that a particular campaign will achieve its objective and assign appropriate volunteers to assist in the process. The volunteering suggestions will be made using a composite score based on campaign traits and volunteer performance.

5.3.9 Email and SMS Notifications

Notification types include:

- E-mail based confirmation of donation receipt.
- Campaign updates.
- Approvals/Rejections from administrative side.
- One-Time Password (OTP) delivery through SMS for donor authentication.
- Event-driven real-time notifications.

5.3.10 Testimonial Moderation

As mentioned in the ML section, testimonials with negative sentiment are flagged automatically. Admins can approve, edit, or reject them before they appear on the public-facing campaign pages.

Chapter 6

System Evaluation and Results

Chapter 6: System Evaluation and Results

6.1 System Evaluation Overview

To ensure that the system operates as it was supposed to, both automated and manual tests were carried out on the system. The test was aimed at verifying the functionality of the different parts of the system such as authentication, donation, volunteers management, testimonials moderation, and API analytics.

6.2 Automated Test Coverage

Both frontend and backend parts of the application have executable test suites. The table below summarizes what each layer validates.

Table 6.1: Implemented Evaluation Layers

Evaluation Layer	**Primary Tools**	**Validation Scope**
Frontend Unit/Component	Vitest, React Testing Library	UI behavior, reducers, and client service logic.
Frontend Contract	MSW	Contract-level checks for API wrapper behavior and payload handling.
Backend Unit/Integration	xUnit, Moq	Controller and service behavior under success and error conditions.
End-to-End	Playwright	End-user flows from login through dashboard and donation paths.

6.3 Functional Validation Scenarios

The most critical business workflows were tested through a mix of API-level checks and manual end-to-end runs:

Table 6.2: Critical Workflow Validation

Workflow	**Validation Type**	**Expected Outcome**
Registration and login	Automated + manual	Valid account creation, token issuance, and protected-route access.
Donation and payment callback	Manual integration	Pending donation updates to completed on successful callback.
Volunteer and reporting modules	API + UI checks	Report submission and status transitions operate correctly.
Testimonial moderation and analytics	API tests	Sentiment analysis and moderation states are persisted and retrievable.

6.4 ML Endpoint Validation

For each ML module, there is an associated API call within the `MLController`. For the purposes of testing, we made calls to each of these end points using sample data:

- **Forecasting:** Predictions of weekly donations along with confidence interval.
- **Sentiment:** Text classification and sentiment score.
- **Donor Churn:** Scored predictions for donor churn.
- **Campaign Success:** Estimated probability of success of campaigns.
- **Anomaly Detection:** Report of any anomalies in donation streams.

6.5 Limitations and What Comes Next

This build satisfies the core functional requirements; however, there are still areas for improvement prior to any actual production deployment. Some of the issues and the subsequent actions to be taken are outlined below:

- **Performance testing:** Testing for performance and scalability was done, but load, soak, and concurrency testing were not performed.
- **Latency and throughput measurement:** Latency budget and throughput metrics have yet to be determined, so performance optimization targets will still remain speculative.
- **Validation dataset size:** The current model was tested using relatively small datasets that should be expanded to verify model generalization.
- **ML model drift detection:** No mechanism for periodic testing and retraining exists, which is crucial for reliable long-term use.
- **Penetration testing:** The basic security measures are implemented, but no penetration testing has been performed yet.
- **Operational readiness:** Basic observability is in place, but structured tracing, threshold-based alerting, and incident handling playbooks have not been developed.
- **Payment resiliency:** Callbacks are managed appropriately, but failure simulation for retries and gateway failure is still incomplete.
- **Data governance:** Retention policies and archival workflow requirements have not been defined formally.

Tackling these issues will transform the current validated prototype into a fully deployable platform with explicit performance and reliability SLAs.

Chapter 7

Conclusion and Future Work

Chapter 7: Conclusion and Future Work

7.1 Overview

This chapter concludes the report with a brief overview of what was developed and its importance. First, it provides an overview of the capabilities provided by the tool, emphasizing its benefits in terms of transparency and efficiency. Then, it mentions the main limitations of the tool after validation, indicating the areas where further analysis of its performance, security, and machine learning capabilities is required. Finally, the chapter describes the improvements that could be made to make it more useful.

7.2 Conclusion

Considering the initial goals in terms of functionality and features to be implemented, it can be noted that the system has developed into quite a complex one. Indeed, it is able to provide users with all the features necessary for any charity campaign on a university campus, from gathering donations to managing finances and recruiting volunteers. Moreover, it has features that no other similar systems have.

A quick summary of what the system delivers:

- **Donations Management:** Secure, instant transactions via SSLCommerz, receipts creation upon completion, and automatic updates for campaign totals.
- **Campaign Transparency:** Real-time tracking of progress and donation, sharing campaign information and a public reserve pool that makes donors know how their money will be spent.
- **Support for Volunteers:** Logging of activities and hours spent, and rewarding regular volunteers with a ranking system.
- **Insights From ML Algorithms:** Forecasting donations, anomalies detection, donor churn scoring, and sentiment analysis within the application backend.

- **Financial Management:** Vouchers should be approved before money transfer, formal withdrawal requests from the account, and full audit tracking.
- **Tracking of Physical Donations:** In-kind donations logging, valuation, and posting to the financial ledger.
- **Moderation of Testimonials:** Sentiment analysis algorithm detects sensitive posts and reports them to admins.
- **RBAC Implementation:** Proper role-based access control allows users from various roles to have limited permissions.
- **Multi-device Compatibility:** Mobile, tablet, and desktop interfaces conform to accessibility guidelines.

The tech stack of ASP.NET Core, React, SQL Server, and Redux Toolkit performed very well. The tech stack has grown mature enough for production use and can scale well. The security features were incorporated at an early stage of development. These include JWT authentication, BCrypt encryption of passwords, server validation, and cross-origin resource sharing restrictions. None of these features are cutting-edge technologies, but they cover all OWASP recommendations and help earn donor confidence. At the very least, what I have learned through this exercise is that the manual way of doing things for campus fundraising can easily be done using software, which would make the process easier, cleaner, and more efficient.

7.3 Future Work

The current system is functional, but there is plenty of room to grow. Some directions worth pursuing:

- **Multilingual Support:** Making the user interface and notifications in Bangla and other languages will enhance the accessibility of the platform. Furthermore, sentiment analysis should be multilingual.

- **Enhanced ML Models:** Although the models used are adequate, experimenting with other datasets and deep learning architectures like neural networks can increase accuracy.
- **Blockchain:** A blockchain system will increase transparency by recording all donations in an immutable ledger.
- **Recommendation Engine:** Donors will likely make more donations when recommended to specific campaigns in line with their previous activities on the site.
- **ERP Integration:** Universities and organizations have enterprise resource planning (ERP) software for accounting purposes. It may be useful to incorporate DMS with those systems and avoid duplicate data entry.
- **Mobile Application:** A native Android and iOS application will increase usability and allow for offline donation management for volunteers.
- **Customizable Queries:** Admins will be able to query the database themselves and export information in any desired format, thus avoiding manual work with Excel.
- **Social Impact Analysis:** The introduction of social return on investment will allow all stakeholders to evaluate the impact of campaigns on society in addition to the financial figures.

7.3.1 Suggested Roadmap

Rolling these out in phases keeps the work manageable:

Table 7.1: Future Enhancement Roadmap

Phase	**Primary Deliverables**	**Expected Outcome**
Phase 1 (0–3 months)	Security hardening, expanded test automation, and observability dashboards.	Improved reliability and reduced regression risk in production releases.
Phase 2 (3–6 months)	Mobile optimization, multilingual support, and reporting enhancements.	Higher accessibility and better stakeholder communication across user groups.
Phase 3 (6–12 months)	Advanced personalization and stronger ML models using larger historical datasets.	Better donor retention and improved forecasting quality.
Phase 4 (12+ months)	External ERP integrations and optional transparency-ledger extensions.	Stronger institutional interoperability and long-term governance maturity.

Each phase builds on the previous one, so the system gets more capable over time without needing a risky “big bang” release.

References

References

[1] David Robinson. Digital fundraising: How technology is transforming charitable giving. *Journal of Nonprofit and Public Sector Marketing*, 31(4):377–392, 2019.

[2] Jutta Sander. The digital transformation of fundraising: Key technology trends and implications for nonprofit organizations. *Nonprofit Management and Leadership*, 32(1):15–35, 2021.

[3] Md Hossain et al. E-payment adoption in bangladesh: A study on consumer behavior. *International Journal of Business and Management*, 2019.

[4] Adam Freeman. *Pro React 18 with TypeScript: Building Enterprise-Grade Applications*. Apress, New York, NY, 2021.

[5] Boris Cherny. *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media, 2019.

[6] Andrew Lock. *ASP.NET Core in Action*. Manning Publications, Shelter Island, NY, 2nd edition, 2021.

[7] Mark Masse. *REST API Design Rulebook: Designing Consistent RESTful Web API Interfaces*. O'Reilly Media, Inc., 2011.

[8] Julie Lerman and Rowan Miller. *Programming Entity Framework Core: Data Access for .NET Applications*. O'Reilly Media, Sebastopol, CA, 2020.

[9] Carlos Coronel and Steven Morris. *Database Systems: Design, Implementation, & Management*. Cengage Learning, 2016.

[10] Ravi S Sandhu, Edward J Coyne, Hal L Feinstein, and Charles E Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

[11] Robert Jones and Sarah Miller. Modern web development stacks for enterprise applications: A comparison of .net, react, and angular. *Journal of Software Engineering and Applications*, 16(4):135–150, 2023.

[12] Lin Wang and Wei Zhang. Enhancing donor engagement and retention through personalized digital experience. In *Proceedings of the International Conference on E-business and E-commerce (ICEC)*, pages 55–62. ACM, 2021.

[13] Christopher Barnes. The rise of crowdfunding and digital fundraising: A nonprofit's guide to success. *Journal of Nonprofit and Public Sector Marketing*, 30(4):430–445, 2018.

[14] Emmanuel Kalinda and Wilfred Mwase. University-based donation platforms: Challenges and opportunities for digital transformation in higher education philanthropy. *Journal of Higher Education Development*, 15(2):88–104, 2022.

[15] As-Sunnah Foundation. As-sunnah foundation: Faith-based humanitarian digital ecosystem, 2025. Faith-based humanitarian organization utilizing mobile financial services and visual documentation for philanthropic transparency within South Asian communities.

[16] Jianping Huang, Yujia Lu, Kai Chen, Min Zheng, and Jinlong Liu. An online donation system based on blockchain and smart contract. In *2020 IEEE International Conference on Services Computing (SCC)*, pages 102–109. IEEE, 2020.

[17] Juan Sanchez, Ricardo Gutierrez, and Isabel Martinez. A conceptual framework for implementing secure and transparent e-donation systems. *International Journal of Computer Science and Information Security*, 17(7):112–120, 2019.

[18] Georg Disterer. Information security management: Iso/iec 27001 standard and its application in software systems. *Journal of Information Security*, 4(2):92–100, 2013.

[19] OWASP Foundation. Owasp top ten web application security risks. *Open Web Application Security Project*, 2021. Available at: https://owasp.org/Top10.

[20] Clayton Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 8, pages 216–225. AAAI Press, 2014.

[21] Alex Banks and Eve Porcello. *Learning React: Functional Web Development with React and Redux*. O'Reilly Media, 2017.

[22] Sanna Suoranta and Antero Toivonen. Otp-based multi-factor authentication: Security challenges and implementation strategies in web applications. *Journal of Internet Security and Privacy*, 7(1):12–28, 2019.

[23] Michael B Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, Internet Engineering Task Force, 2015. Available at: https://tools.ietf.org/html/rfc7519.

[24] Niels Provos and David Mazieres. A future-adaptable password scheme. In *USENIX Annual Technical Conference Proceedings*, pages 81–91. USENIX Association, 1999.

[25] Adam Barth, Collin Jackson, and John C Mitchell. Robust defenses for cross-site request forgery. In *Proceedings of the 15th ACM conference on Computer and communications security*, pages 75–88, 2008.

[26] Wei Chen and Xiaoming Liu. Machine learning applications in nonprofit fundraising: Predictive analytics for donor behaviour. *Journal of Nonprofit Technology and Innovation*, 8(2):45–67, 2022.

[27] Priya Gupta and Amit Sharma. Predicting donor attrition in charitable organizations using supervised machine learning. *International Journal of Data Science and Analytics*, 10(3):211–225, 2020.

[28] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Martin Fowler, et al. Manifesto for agile software development. *Agile Alliance*, 2001. Available at: https://agilemanifesto.org.

[29] Jim Highsmith and Martin Fowler. The agile manifesto. *Software Development Magazine*, 9(8):29–30, 2001.

[30] Hossein Hassani. Singular spectrum analysis: methodology and comparison. *Journal of Data Science*, 5(2):239–257, 2007.